Principles of Robot Autonomy I

Robotic Sensors, Point cloud alignment



Stanford University



SIA Lab
Safe and Intelligent Autonomy Lab





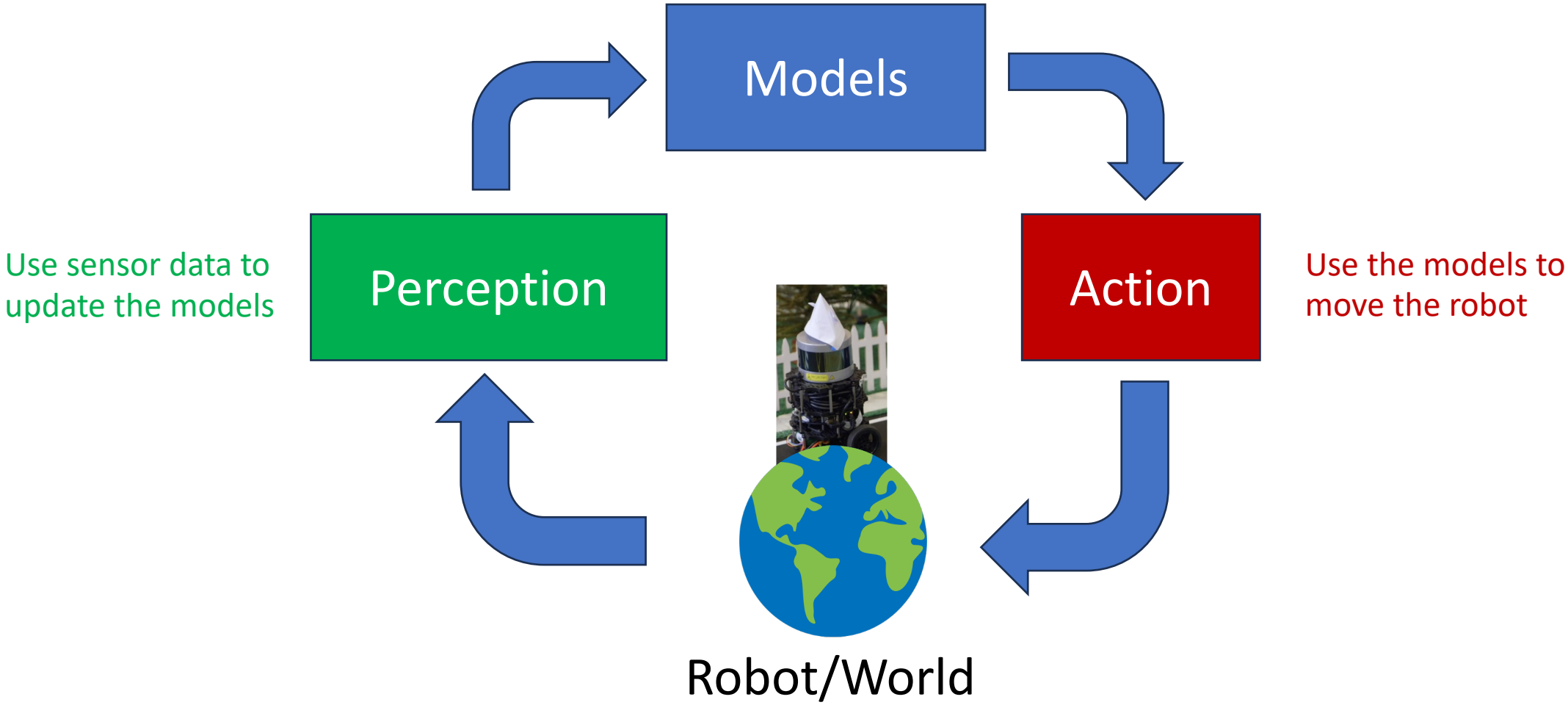
IPRL



Logistics

- Homework 2: due **Friday, Oct 16, at 11:59pm**.
- Homework 3: out Thurs, Oct 15, due Tues, Oct 27
- Midterm: Thursday, Oct 29, in-class
- OAE letters – Please email them to the teaching team asap on cs237a-aut2627-staff@lists.stanford.edu
- Section 6 has been moved to the week of 11/01
- Please consider moving to smaller sections if you are in one of the big sections (Mon, 5-7pm; Wed, 4:30-6:30pm). Email the teaching team.
- Lecture 8:
  - Robotic sensors
  - Point cloud alignment, ICP
  - Pinhole camera model, camera calibration

Full Stack Autonomy: the Perception-Action Loop

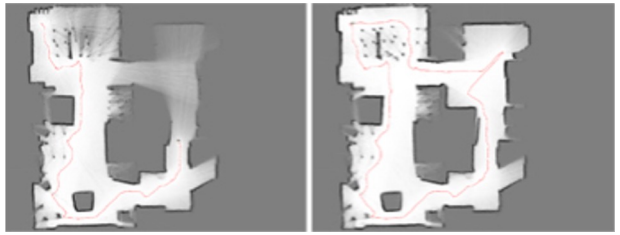


Perception Stack: Computer vision, filtering, SLAM

Use sensor data to update the models

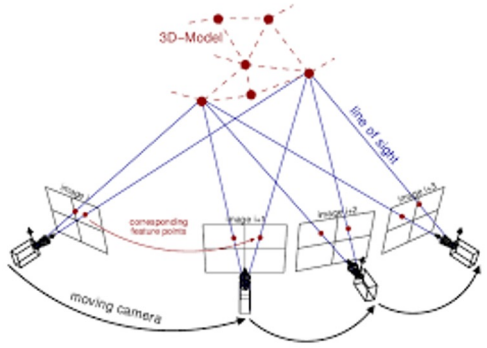
Localization, Mapping, Tracking:

- EKF/Monte Carlo localization
- Occupancy grid mapping
- Pose graph optimization
- Tracking (EKF and Particle Filter)
- **AA273: Filtering (Schwager)**
- **AA275: Navigation (Gao)**



Information extraction

- Computer vision: features, correspondences, Structure from Motion (SfM), depth
- Lidar scan matching, ICP
- **CS231A: Comp Vision**

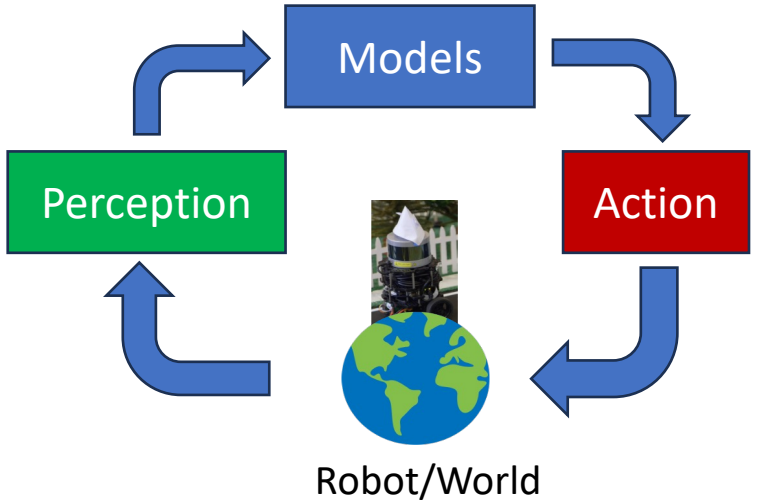


Sensors:

- RGB Cameras, RGB-D/stereo cameras, Lidar
- IMU, GPS, wheel encoders



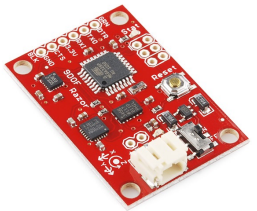




Sensors

Sensors

- Proprioceptive: IMU, joint and wheel encoders. GPS
- Exteroceptive: RGB, stereo, RGB-D, Lidar



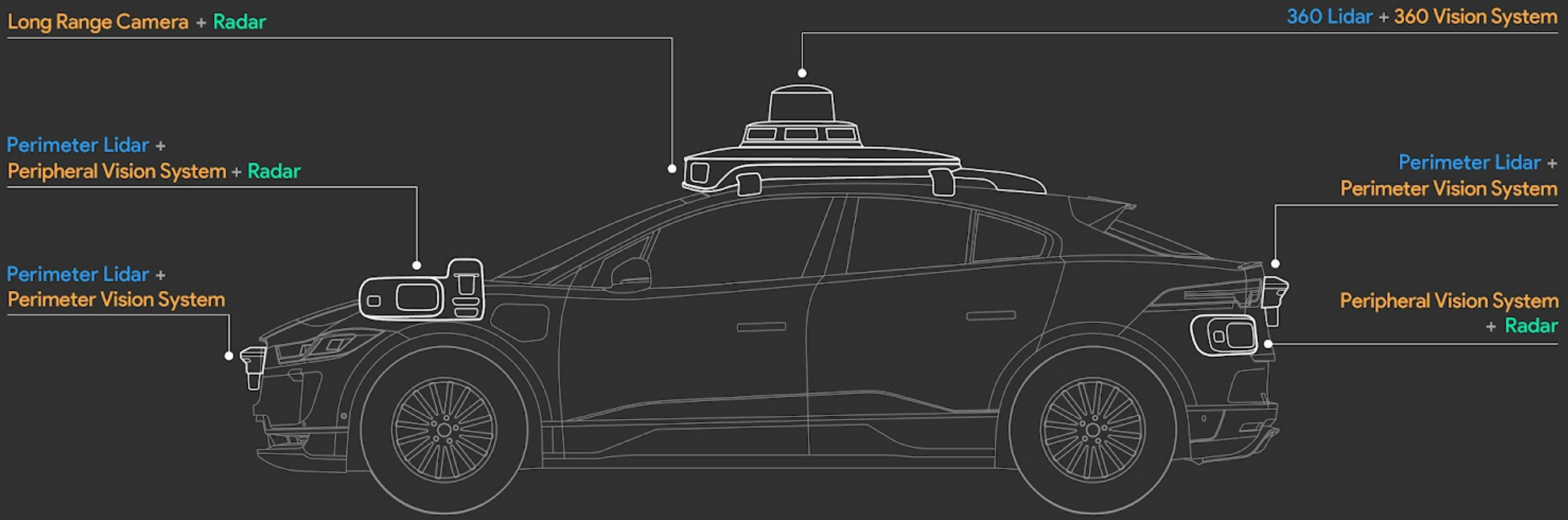


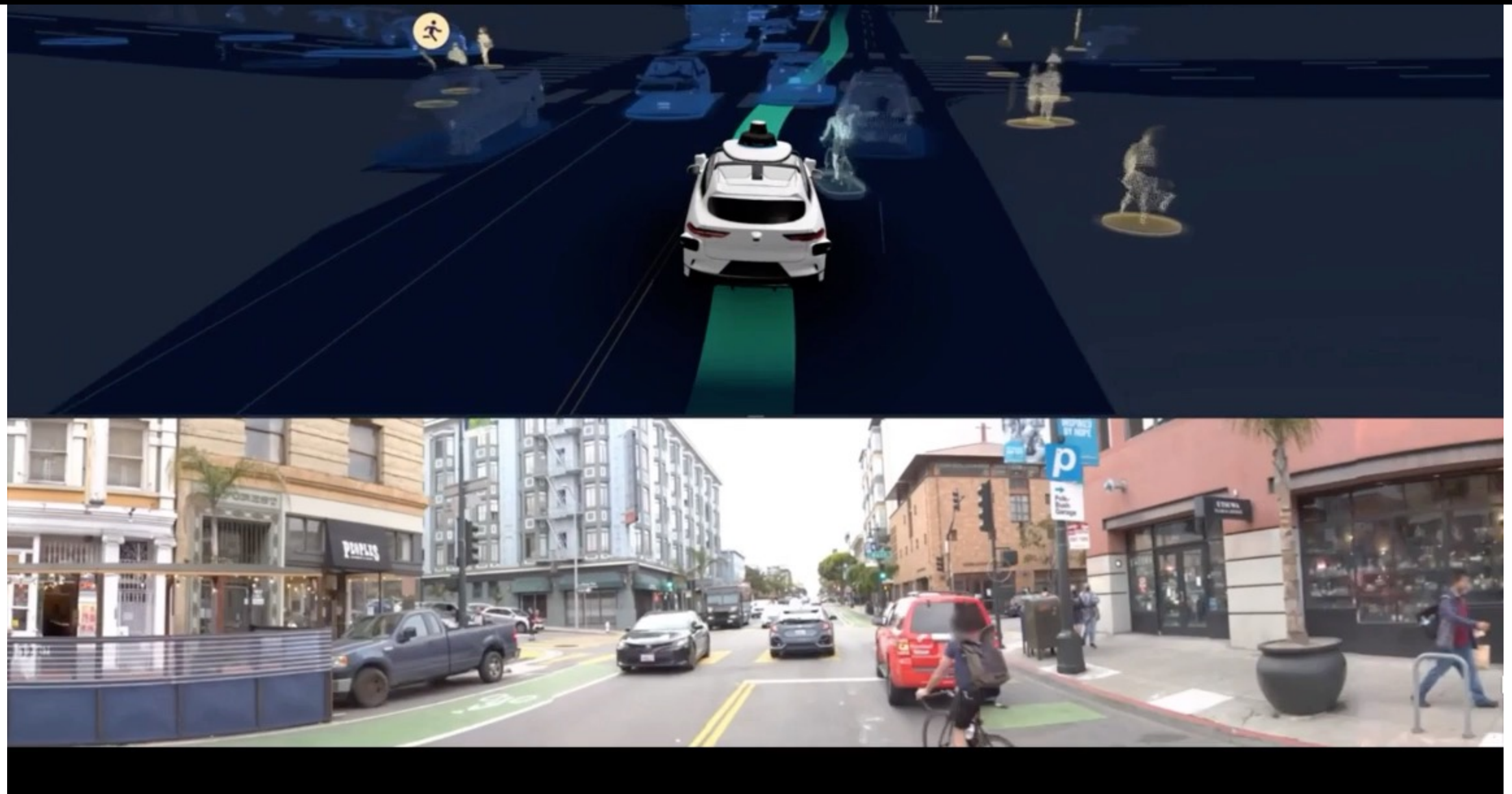






Example: self-driving cars





Classification of sensors

- Proprioceptive: measure values internal to the robot
  - E.g.: robot acceleration, robot arm joint angles, and battery voltage
- Exteroceptive: acquire information from the robot's environment
  - E.g.: distance measurements and light intensity

- Passive: measure ambient environmental energy entering the sensor
  - Challenge: performance heavily depends on the environment
  - E.g.: temperature probes and cameras
- Active: emit energy into the environment and measure the reaction
  - Challenge: might affect the environment
  - E.g.: ultrasonic sensors and laser rangefinders

An ecosystem of sensors

- Joint and Wheel Encoders
- IMU
- GPS

Proprioceptive: non-navigational ego-states

- RGB
- RGB-D
- Lidar
- Sonar
- Radar

Exteroceptive: relative poses of map features
color of map features
relative poses of targets
color of targets

Proprioceptive: Encoders

- Encoder: Measures angle of a wheel or joint.

- Mechanism: light emitter/receptor interrupted by black lines on a disk, resulting pulses indicate angle of rotation.

- Application:
  - Approximate robot localization
  - Called wheel odometry or “dead reckoning”



Wheel encoder
Credit: Pololu

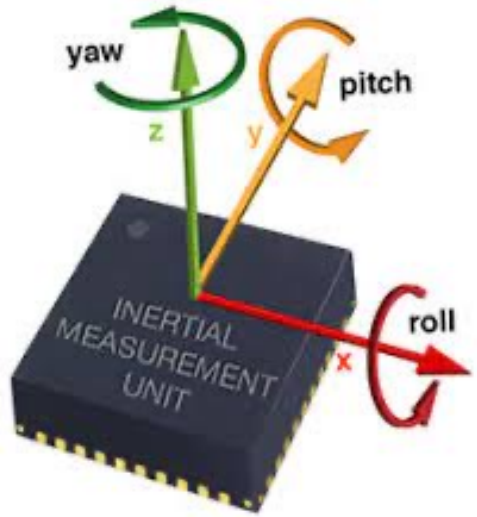


Credit: Honest Sensor

Proprioceptive: Inertial Measurement Unit (IMU)

- Rate Gyroscope: measures angular velocity vector in body-fixed frame

$$y_\omega = (\omega_x, \omega_y, \omega_z) + \nu_\omega$$ ← noise

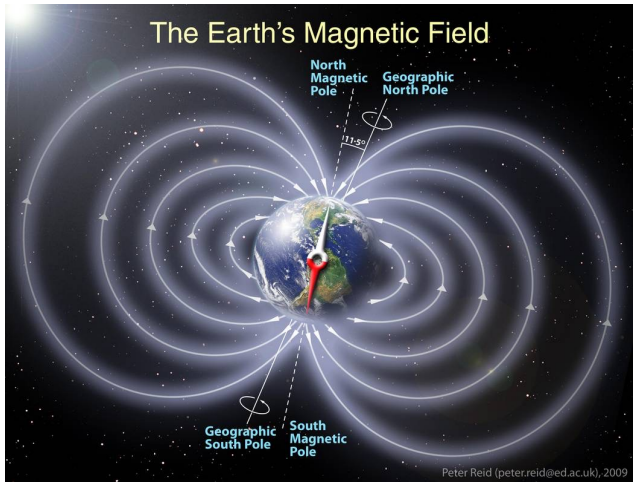


- Accelerometer: measures linear acceleration vector in body-fixed frame (including gravity)

$$y_a = (a_x, a_y, a_z) + g + \nu_a$$

- Magnetometer: measures local magnetic field. Earth's + other magnetic disturbances. Compass.

$$y_B = (B_x, B_y, B_z) + \nu_B$$



IMU often pre-processed to get orientation

- Orientation: Low pass filter accel to get gravity vector (global down). Cross with magnetometer to get East. Cross East with down to get north.

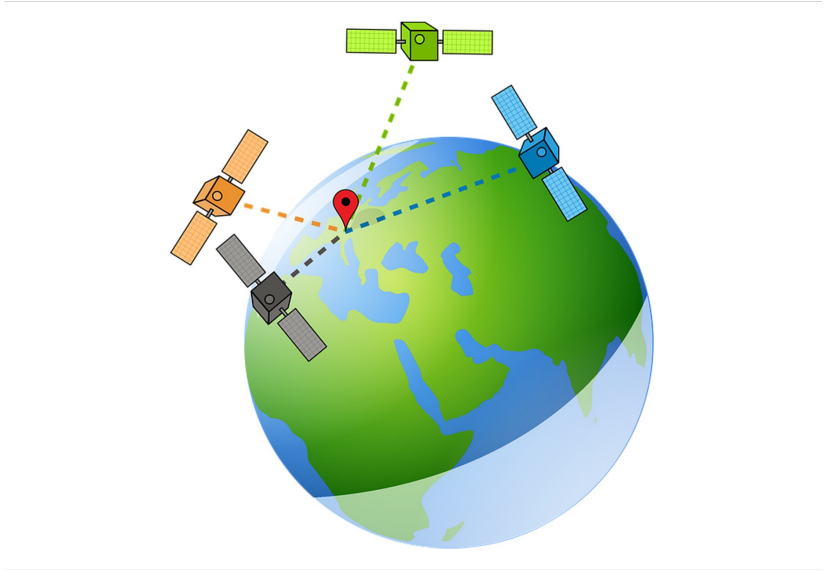
$$y_a = (a_x, a_y, a_z) + g + \nu_a \qquad g \leftarrow \text{Filter}(y_a)$$

$$e_{\text{Down}} = \frac{g}{\|g\|} \qquad e_{\text{East}} = \frac{e_{\text{Down}} \times y_B}{\|e_{\text{Down}} \times y_B\|} \qquad e_{\text{North}} = \frac{e_{\text{East}} \times e_{\text{Down}}}{\|e_{\text{East}} \times e_{\text{Down}}\|}$$

- NED frame rotation matrix world-to-body:

$$R = [e_{\text{North}} \;\; e_{\text{East}} \;\; e_{\text{Down}}]$$

Proprioceptive: GPS (more generally, GNSS)



- Multi-lateration: use time-of-flight to get distance, known satellite locations to get end points. Then solve for receiver location with multi-lateration:

$$y_{\text{GPS}} = (q_x, q_y, q_z) + \nu_{\text{GPS}}$$

Exteroceptive: Active ranging

- Provide direct measurements of distance to objects in vicinity
- Key elements for both localization and environment reconstruction
- Main types:
  1. Time-of-flight active ranging sensors (e.g., ultrasonic and laser rangefinder)



Credit: https://electrosome.com/hc-sr04-ultrasonic-sensor-pic/

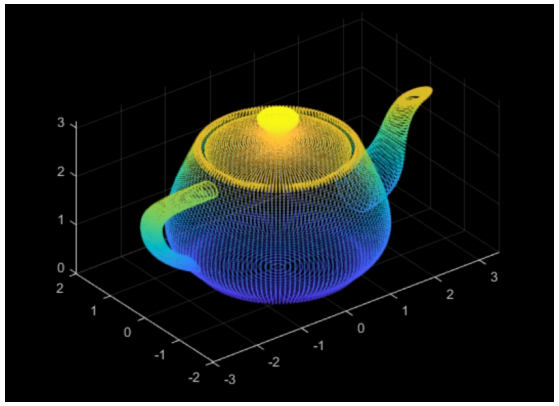


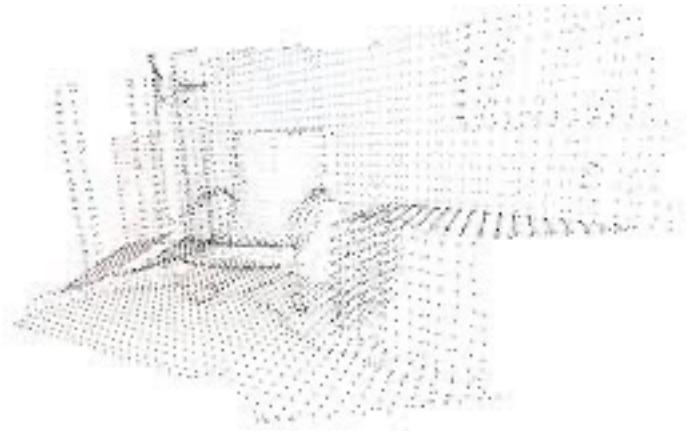


  2. Geometric active ranging sensors (optical triangulation and structured light)

Point clouds

- All these sensors output point cloud data

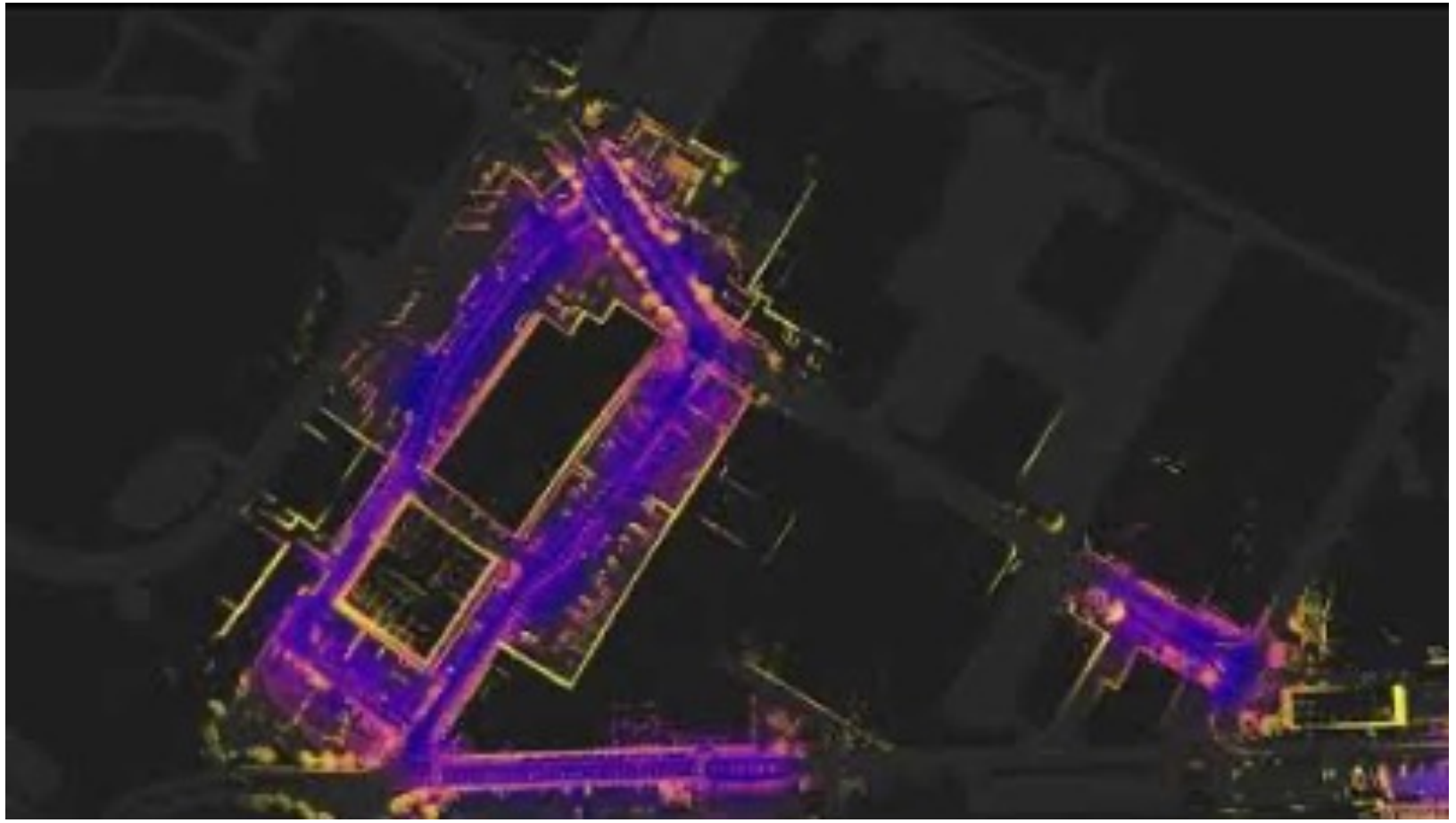




$$P = (p_1, p_2, \ldots, p_N), \quad p_i = (p_x, p_y, p_z)^T$$

Example Lidar data from nuScenes dataset

https://www.nuscenes.org/

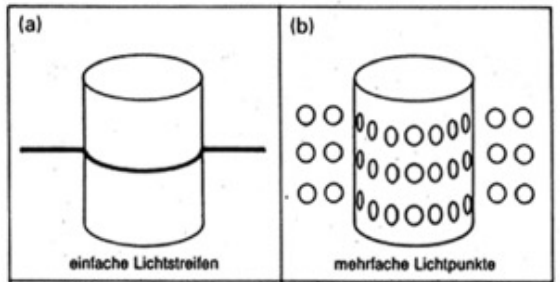


Lidar point clouds overlayed with Camera images https://www.nuscenes.org/



Geometric active ranging

- Fundamental principle: use geometric properties in the measurements to establish distance readings
- The sensor projects a known light pattern (e.g., point, line, or texture); the reflection is captured by a receiver and, together with known geometric values, range is estimated via triangulation
- Examples:
  - Optical triangulation (1D sensor)
  - Structured light (2D and 3D sensor)

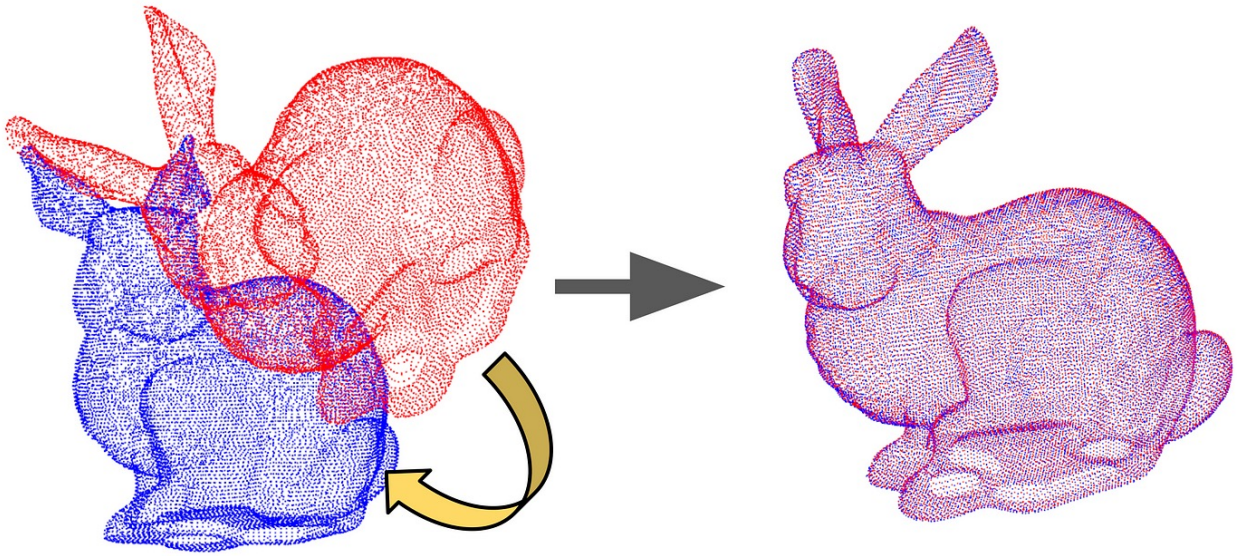






Credit: Matt Fisher

Classic Problem: Point Cloud Alignment



- Point cloud alignment: Align point cloud A (from a new scan) to point cloud B (from a previous scan or map)
- Obtain the relative pose: The pose transform required to move P_A to align with P_B

$$(\tau_{BA}, R_{BA})$$

Translation vector

Rotation matrix

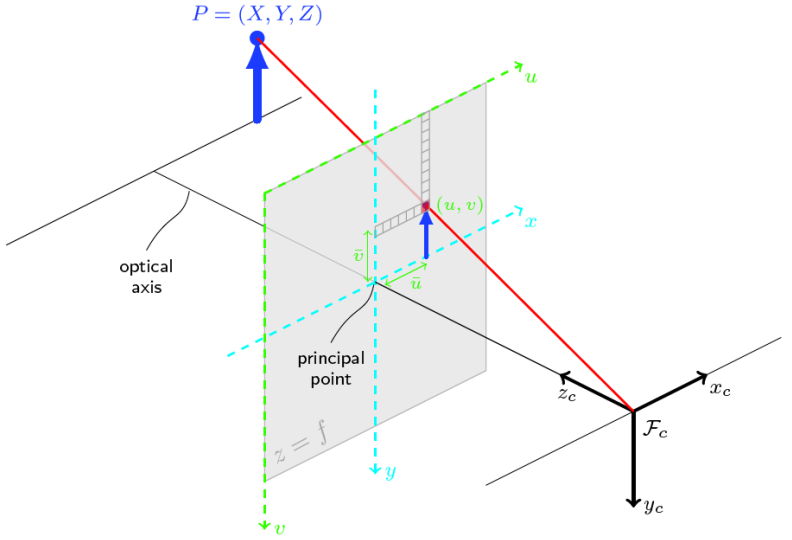
RGB (monocular vision)



- Produces a matrix of RGB color vectors

$$I = \begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1N} \\ c_{21} & c_{22} & \cdots & c_{2N} \\ \vdots & \ddots & & \vdots \\ c_{M1} & \cdots & & c_{MN} \end{bmatrix}$$

$$c_{uv} = (r_{uv}, g_{uv}, b_{uv})$$



- Fundamental problem: Where in the world frame was the "thing" that produced pixel c_{uv}. How to go from (u, v) pixel coordinates to (p_x, p_y, p_z) world coordinates?

Calibration matrix:

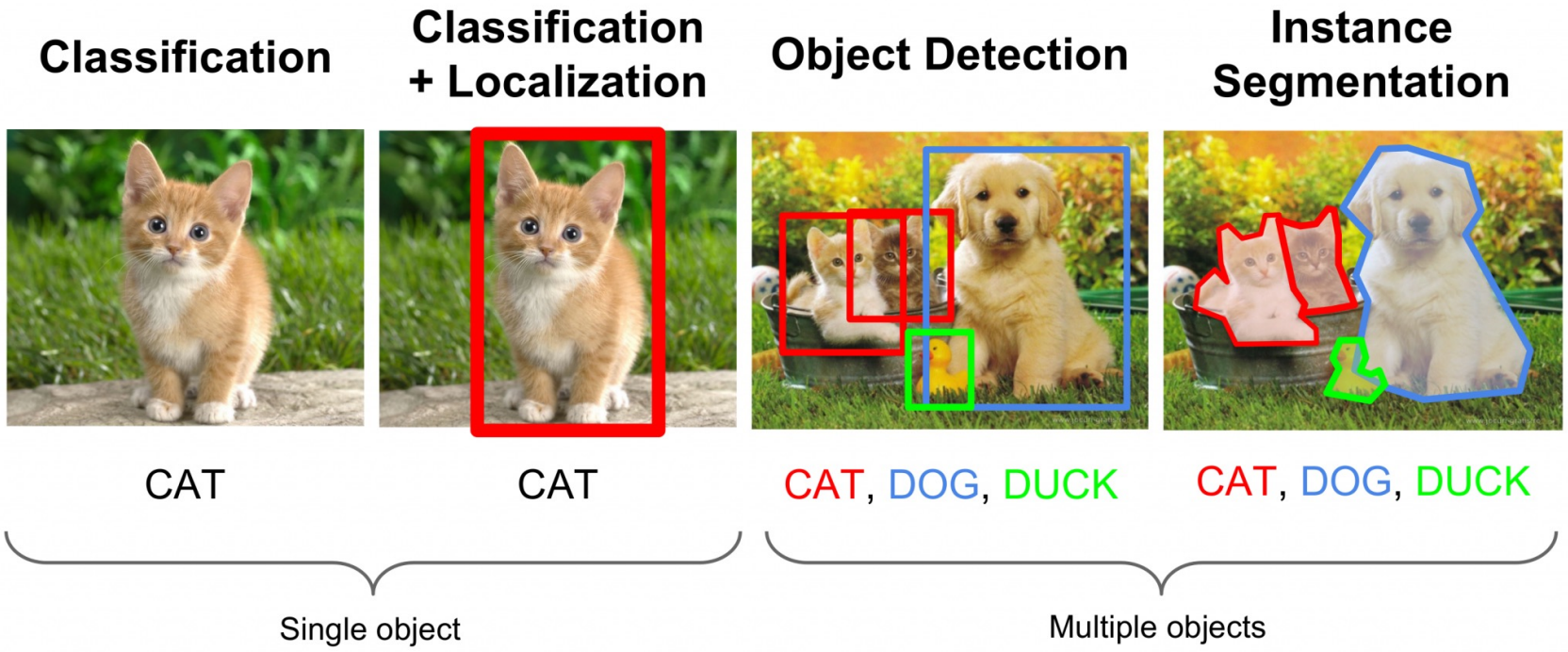
$$K = \begin{bmatrix} \alpha & 0 & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

Pinhole camera model:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}$$

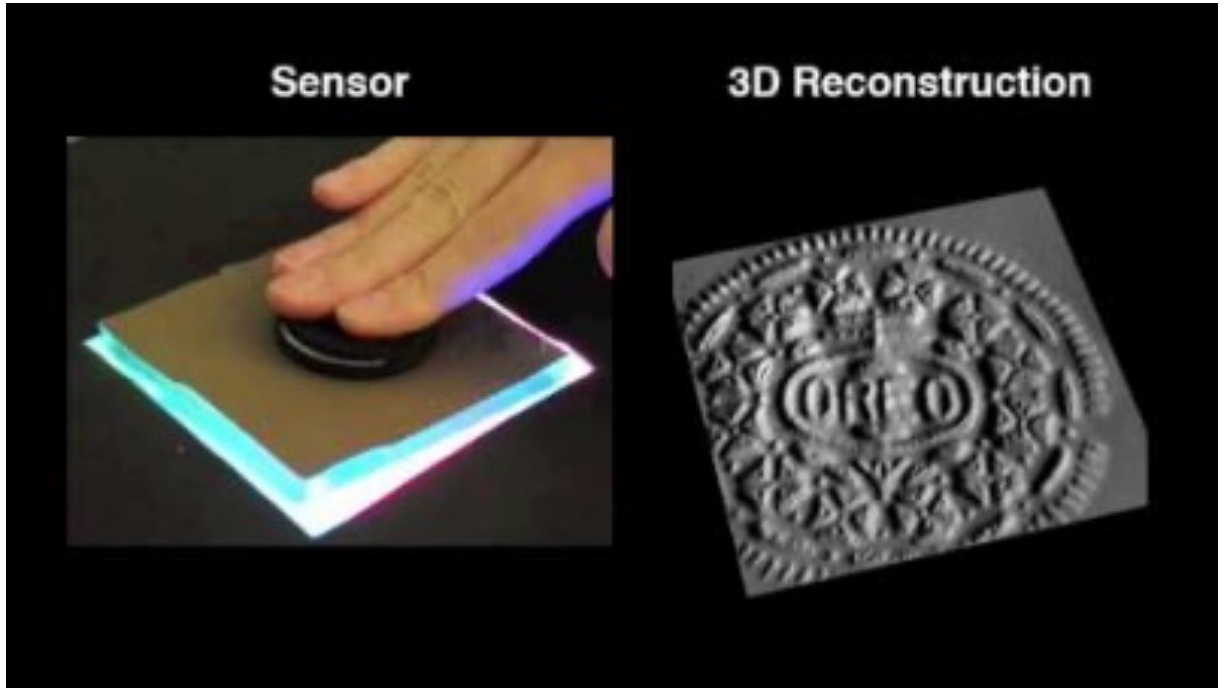
Classic Vision Problems: Semantic

- Classification: What's in the image?
- Detection: Draw bounding boxes around objects with labels
- Segmentation: Identify pixels (masks) belonging to object classes



Several other sensors are available

- Classical, e.g.:
  - Radar (possibly using Doppler effect to produce velocity data)
  - Tactile sensors, load cells
- Emerging technologies:
  - Artificial skins
  - Neuromorphic cameras



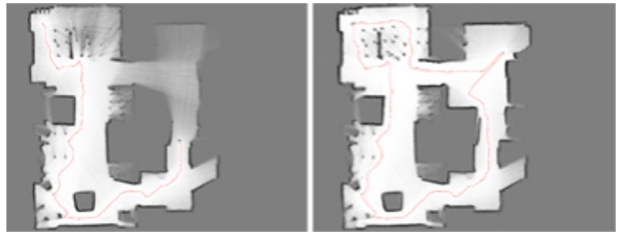
GelSight at Emerging Technologies at SIGGRAPH 2009
https://www.gelsight.com/

Perception Stack: Computer vision, filtering, SLAM

Use sensor data to update the models

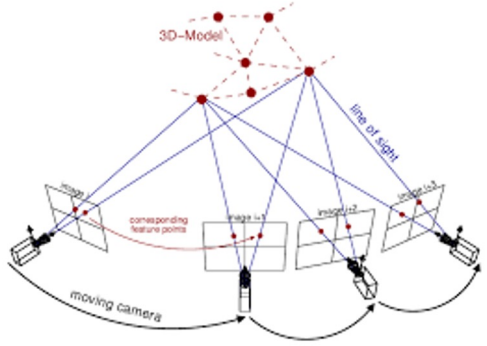
Localization, Mapping, Tracking:

- EKF/Monte Carlo localization
- Occupancy grid mapping
- Pose graph optimization
- Tracking (EKF and Particle Filter)
- **AA273: Filtering (Schwager)**
- **AA275: Navigation (Gao)**



Information extraction

- Computer vision: features, correspondences, Structure from Motion (SfM), depth
- Lidar scan matching, ICP
- **CS231A: Comp Vision**

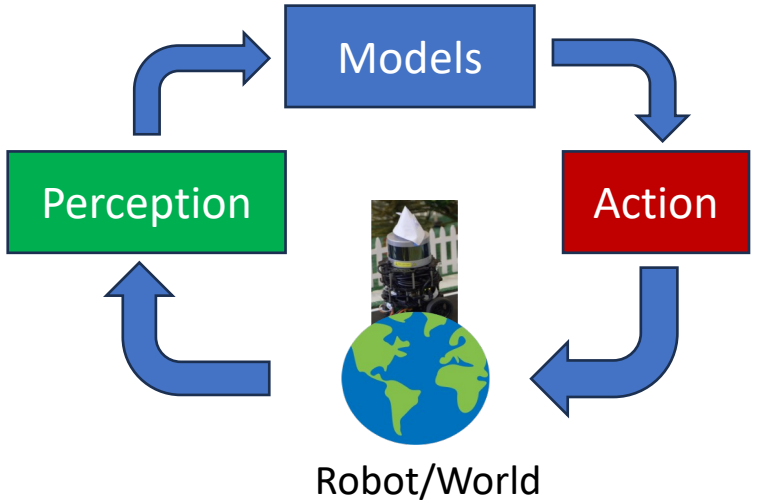


Sensors:

- RGB Cameras, RGB-D/stereo cameras, Lidar
- IMU, GPS, wheel encoders

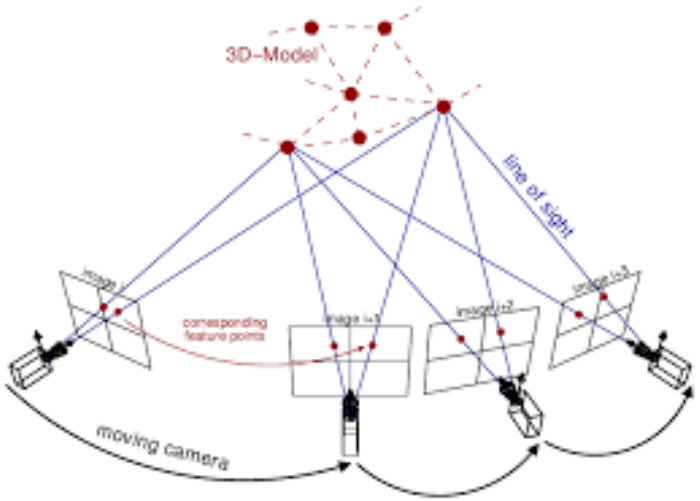




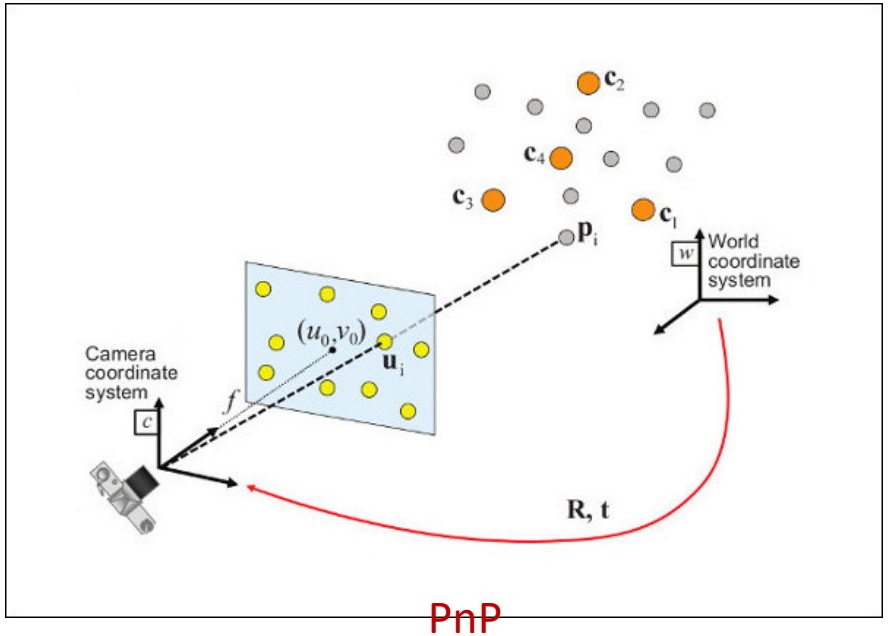


Classic Vision Problems: Geometric

- Structure from Motion (SfM, multi-view geometry): Find 3D locations of points in scene based on pairs or collections of images from different perspectives. Gives a point cloud and camera poses.
- N Point Perspective (PnP): Match 2D points in the image with 3D points in a pre-existing 3D model

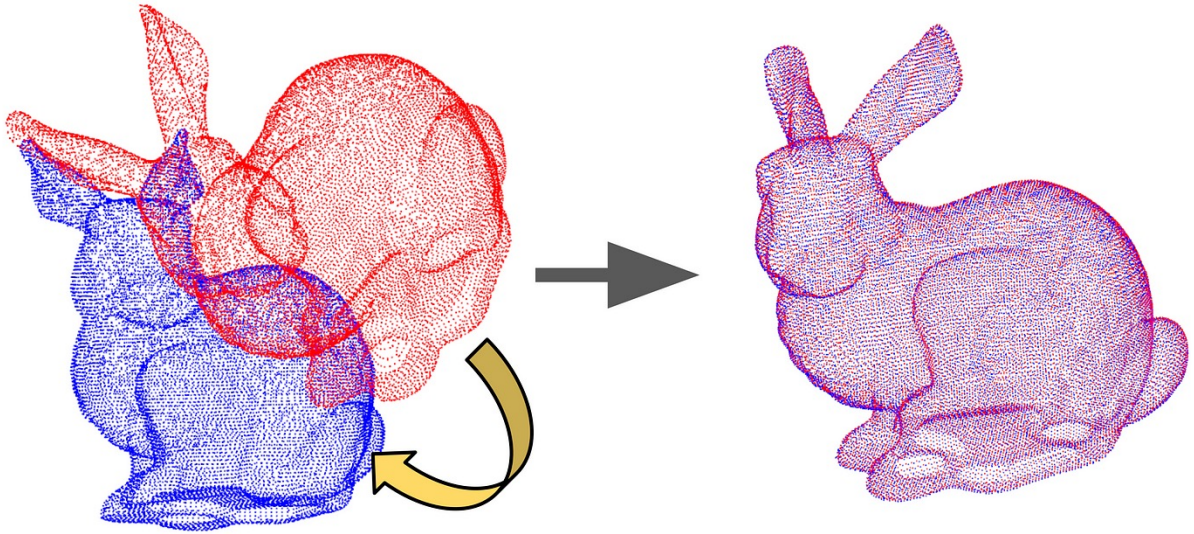


SfM



PnP

Point cloud alignment: Iterative Closest Point (ICP)



- Obtain the relative pose: The pose transform required to move P_A to align with P_B

(τ_{BA}, R_{BA})

Translation vector

Rotation matrix

Basics and notation:

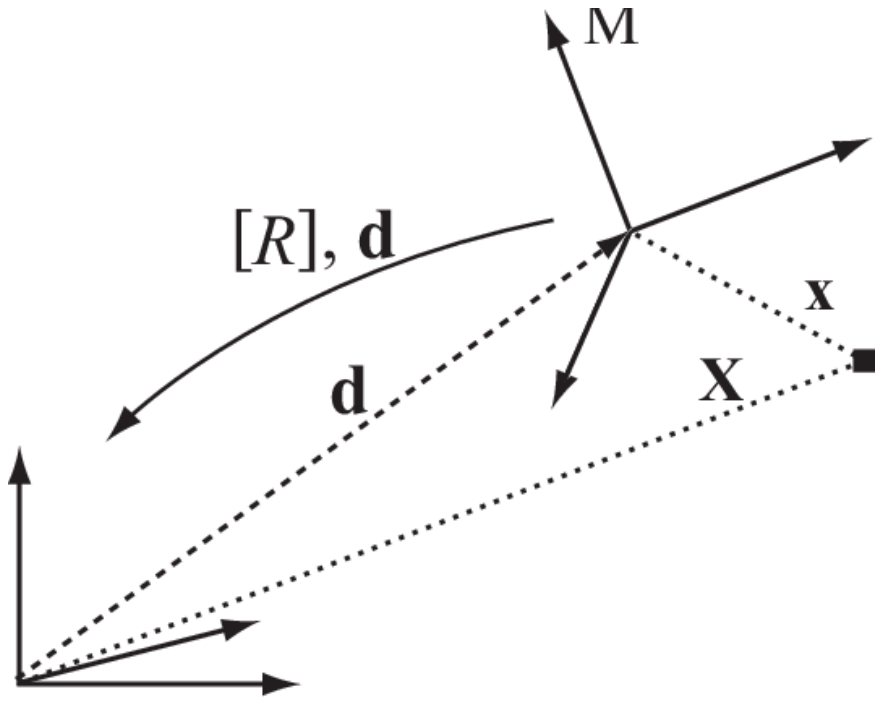
- Rigid body transforms, SE(2), SE(3)

$$(\tau_{BA}, R_{BA}) \in \mathrm{SE}(2) \quad \text{or} \quad \mathrm{SE}(3)$$

$$P_B = R_{AB} P_A + \tau_{BA}$$

rotate

translate



Alignment with Known Correspondences

- Find (τ_{BA}, R_{BA})

$$P_B = R_{AB} P_A + \tau_{BA}$$



$$\min_{(\tau_{BA}, R_{BA})} \|P_B - (R_{BA} P_A + \tau_{BA})\|_{\mathcal{F}}^2$$
$$\text{s.t.} \quad R_{BA}^T R_{BA} = I$$

- Known solution based on SVD, sometimes called Wahba's problem

Singular Value Decomposition Review

- For any n x m matrix M (including singular and rectangular matrices):

$$M = U\Sigma V^T$$

n x n orthogonal
Left singular vectors

n x m diagonal
singular values
(non-negative)

m x m orthogonal
Right singular vectors

E.g. for n > m:

$$U = \begin{bmatrix} u_1 & u_2 & \cdots & u_n \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & 0 & \vdots \\ \vdots & \vdots & \ddots & 0 \\ 0 & 0 & \cdots & \sigma_m \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}$$

largest

smallest

$$V^T = \begin{bmatrix} v_1^T \\ v_2^T \\ \vdots \\ v_m^T \end{bmatrix}$$

- Right null space
- Left null space
- Column space
- Row space

Singular Value Decomposition Review

- For any n x m matrix M (including singular and rectangular matrices):

$$M = U\Sigma V^T$$

E.g. for n < m:

largest

$$\underbrace{\begin{bmatrix} u_1 & u_2 & \cdots & u_n \end{bmatrix}}_{U} \underbrace{\begin{bmatrix} \sigma_1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & \sigma_2 & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \sigma_n & 0 & \cdots & 0 \end{bmatrix}}_{\Sigma} \underbrace{\begin{bmatrix} v_1^T \\ v_2^T \\ \vdots \\ v_m^T \end{bmatrix}}_{V^T}$$

smallest

- Right null space
- Left null space
- Column space
- Row space

Least Norms Solution (n > m):

$$\min_p \|Mp\| \quad \text{subject to} \quad \|p\| = 1 \implies \|U\Sigma V^T p\| \implies \| \begin{bmatrix} u_1 & u_2 & \cdots & u_n \end{bmatrix} \begin{bmatrix} \sigma_1 v_1^T \\ \sigma_2 v_2^T \\ \vdots \\ \sigma_m v_m^T \\ 0 \\ \vdots \\ 0 \end{bmatrix} p \|$$

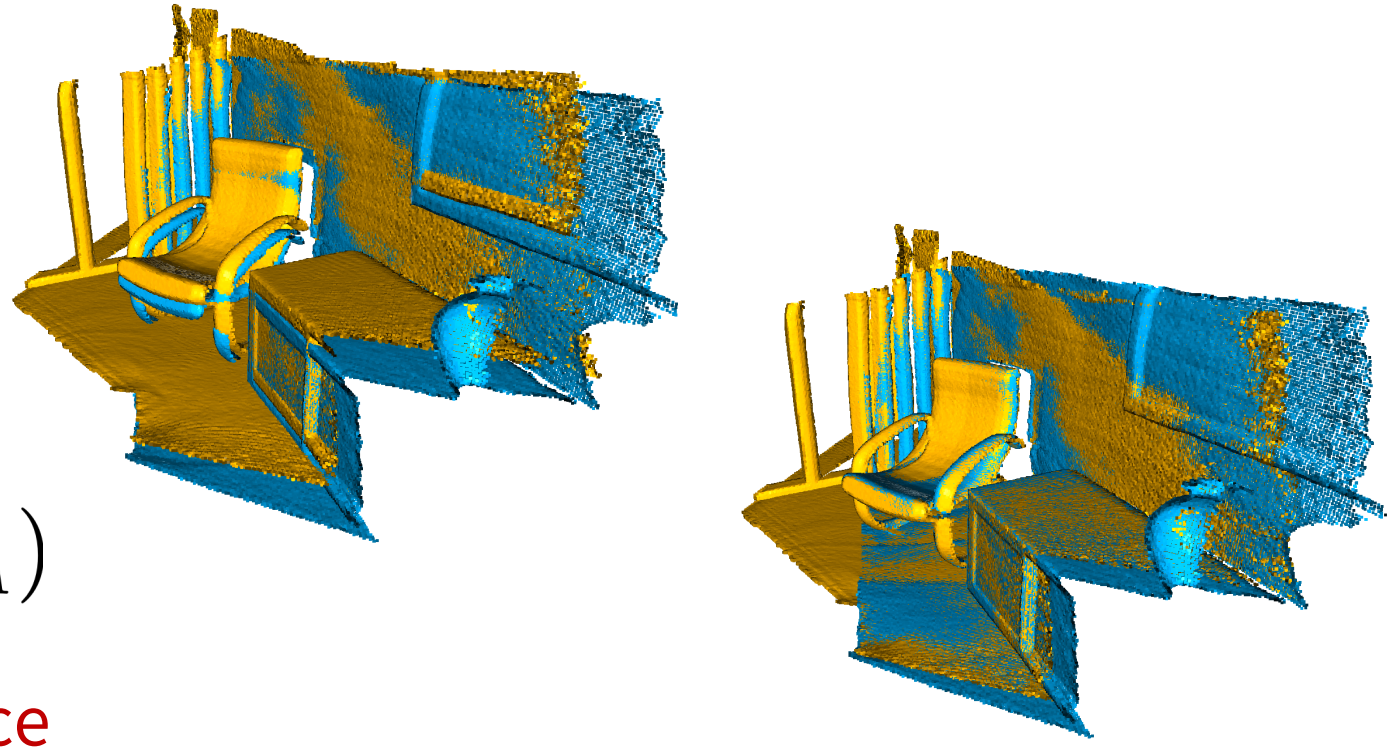
Guess: $p = v_m$

$$\implies \| \begin{bmatrix} u_1 & u_2 & \cdots & u_n \end{bmatrix} \begin{bmatrix} \sigma_1 v_1^T v_m \\ \sigma_2 v_2^T v_m \\ \vdots \\ \sigma_m v_m^T v_m \\ 0 \\ \vdots \\ 0 \end{bmatrix} p \| = \| \begin{bmatrix} u_1 & u_2 & \cdots & u_n \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ \vdots \\ \sigma_m \\ 0 \\ \vdots \\ 0 \end{bmatrix} p \| = \|u_m \sigma_m\| = \sigma_m$$

Solution: $p = v_m$ Gives: $\|Mp\| = \sigma_m$ Smallest singular value

ICP: iterate with closest point heuristic

- For each point in P_A, p_i^A match to closest point p_j^B from P_B

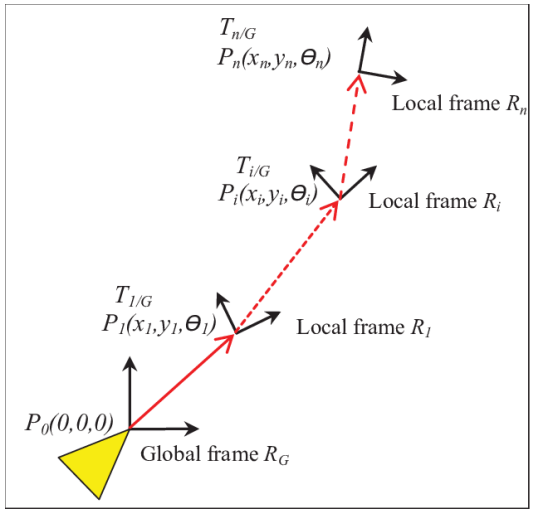


- Solve for (τ_{BA}, R_{BA})
- Repeat until convergence

Images: Open3D

Why? Simultaneous Localization and Mapping (SLAM)

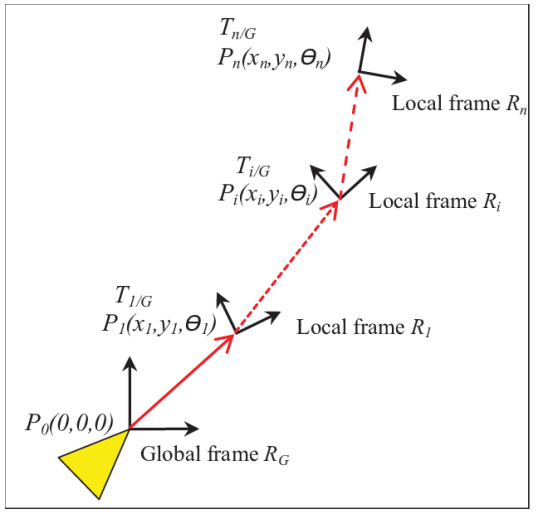
- Lidar odometry: Gives relative poses for robot between successive Lidar scans.
- Pose graph: These Graph where nodes are unknown global poses, and edges are relative pose measurements (e.g., from ICP)
- SLAM front end: constructing the pose graph from raw sensor measurements
- SLAM back end: pose graph optimization (PGO) to find self-consistent set of global robot poses and corresponding global map





Computer Vision

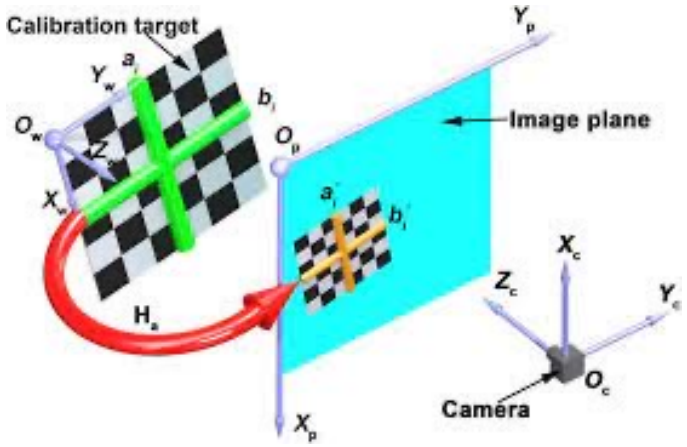
- We can also build a pose graph from an RGB camera!
- Or from a stereo camera
- Or from an RGB-D camera
- First we need a mathematical model of the camera



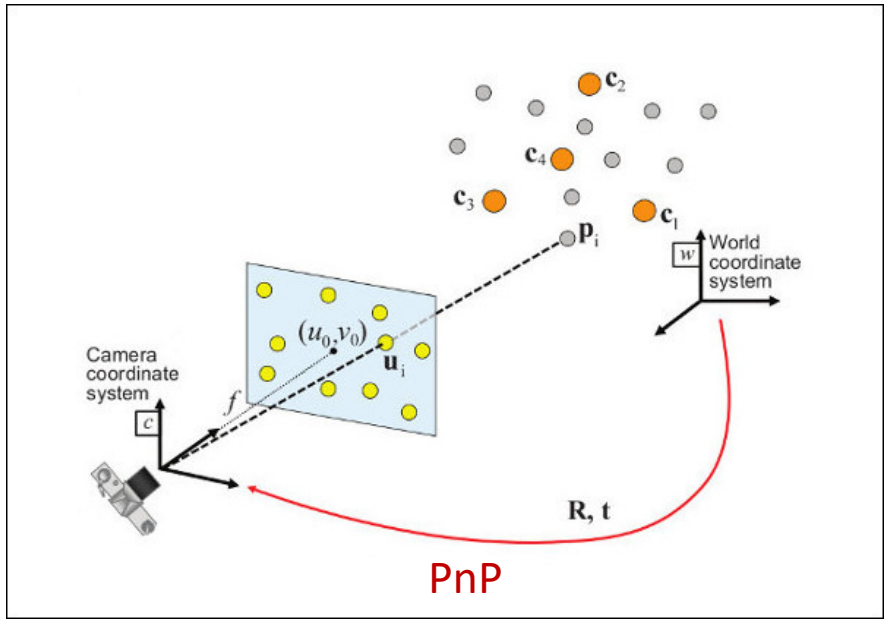


Classic Vision Problems: Geometric

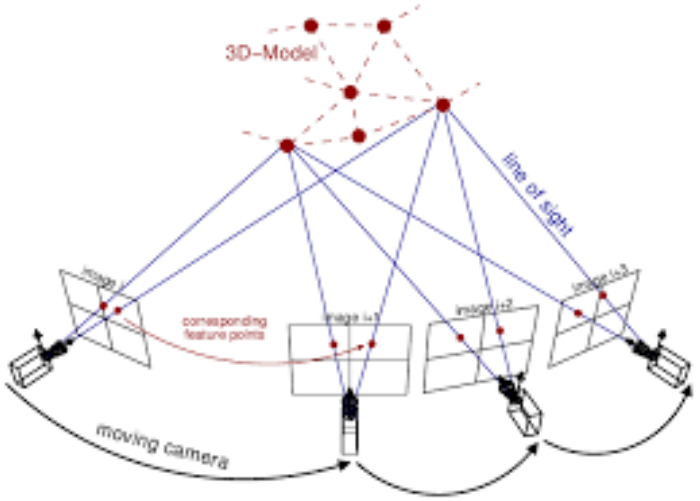
- Camera calibration: Find calibration matrix (intrinsic parameters) and camera pose (extrinsic parameters)
- N Point Perspective (PnP): Match 2D points in the image with 3D points in a pre-existing 3D model
- Structure from Motion (SfM, multi-view geometry): Find 3D locations of points in scene based on pairs or collections of images from different perspectives. Gives a point cloud and camera poses.



Camera calibration



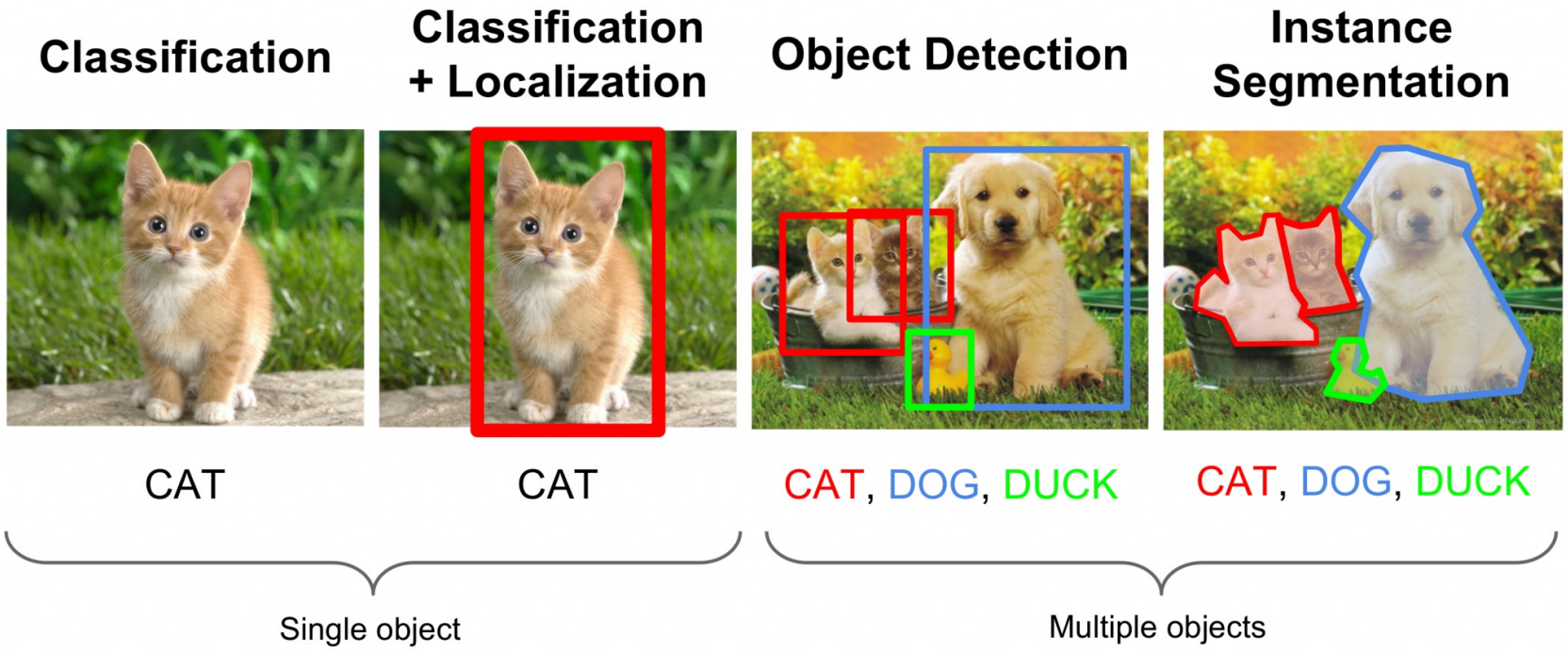
PnP



SfM

Classic Vision Problems: Semantic

- Classification: What's in the image?
- Detection: Draw bounding boxes around objects with labels
- Segmentation: Identify pixels (masks) belonging to object classes



Camera models and camera calibration

- Aim
  - Understand the pinhole camera model and projective geometry
  - Learn about camera calibration and the PnP problem

- Readings
  - SNS: 4.2.3
  - D. A. Forsyth and J. Ponce [FP]. Computer Vision: A Modern Approach (2nd Edition). Prentice Hall, 2011. Chapter 1.
  - R. Hartley and A. Zisserman [HZ]. Multiple View Geometry in Computer Vision. Academic Press, 2002. Chapter 6.1.
  - Z. Zhang. A Flexible New Technique for Camera Calibration. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2000.

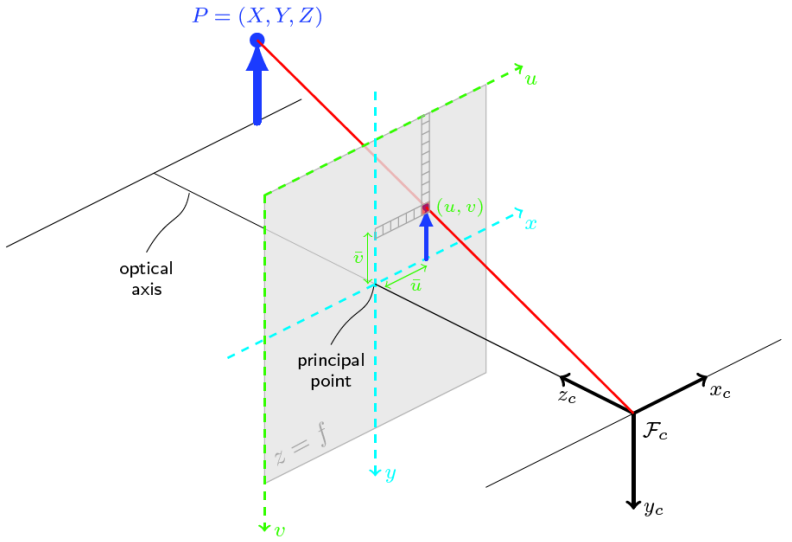
Pinhole camera model



- Produces a matrix of RGB color vectors

$$I = \begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1N} \\ c_{21} & c_{22} & \cdots & c_{2N} \\ \vdots & \ddots & & \vdots \\ c_{M1} & \cdots & & c_{MN} \end{bmatrix}$$

$$c_{uv} = (r_{uv}, g_{uv}, b_{uv})$$



Calibration matrix:

$$K = \begin{bmatrix} \alpha & 0 & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

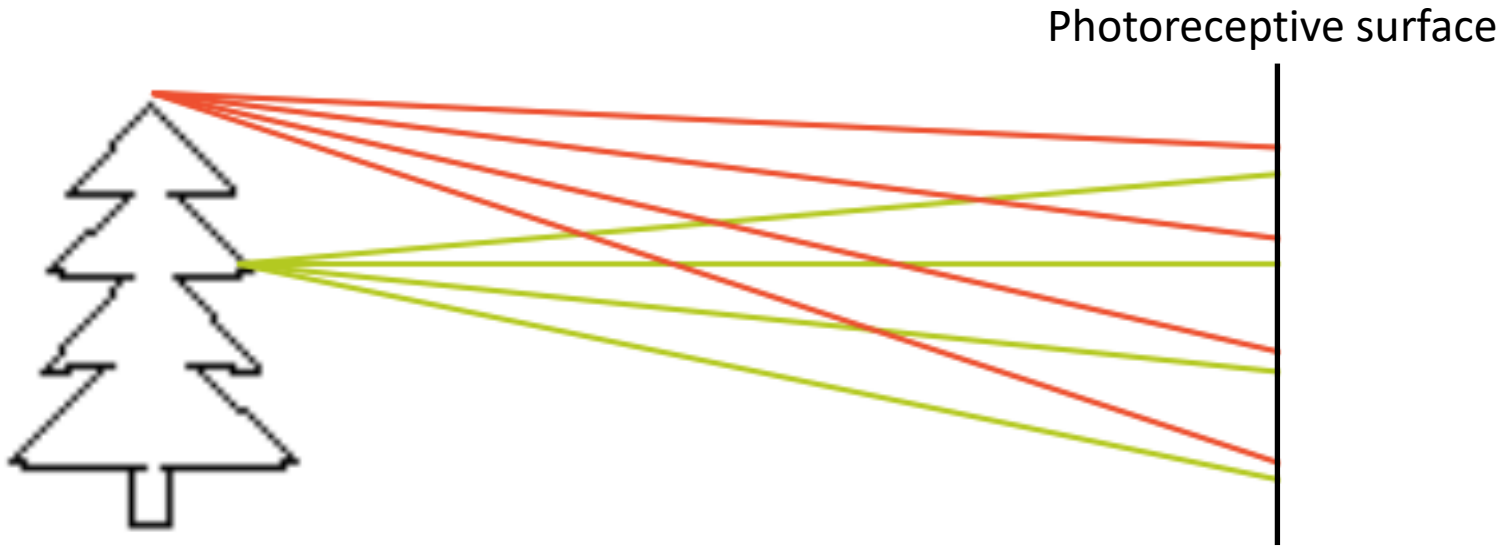
Pinhole camera model:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}$$

- Projection mapping from the "thing" in the world at location (p_x, p_y, p_z) to the pixel coordinates (u, v) that have color c_{uv} in the image.

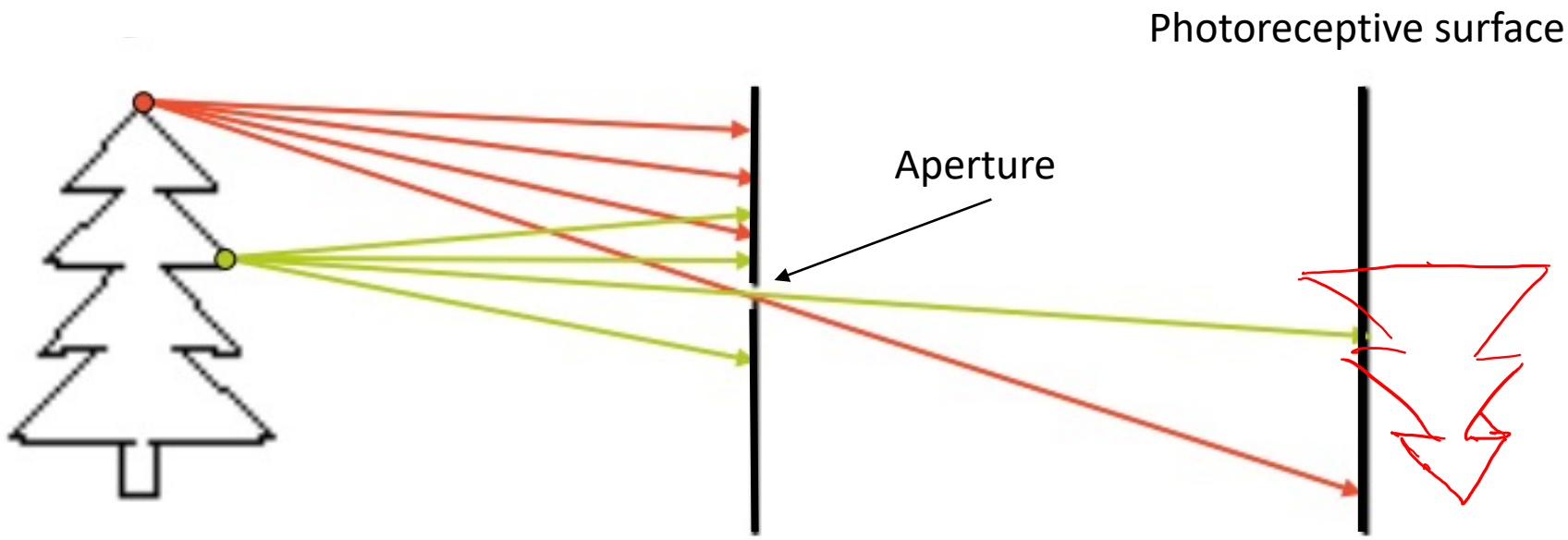
How to capture an image of the world?

- Light is reflected by the object and scattered in all directions
- If we simply add a photoreceptive surface, the captured image will be extremely blurred



Pinhole camera

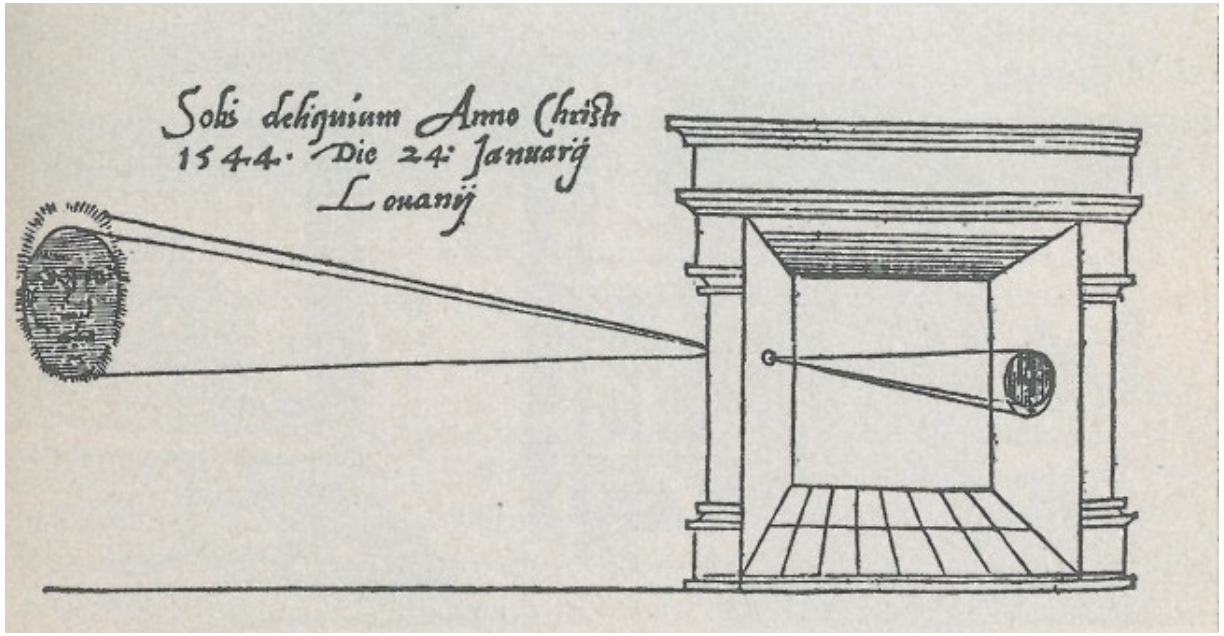
- Idea: add a barrier to block off most of the rays



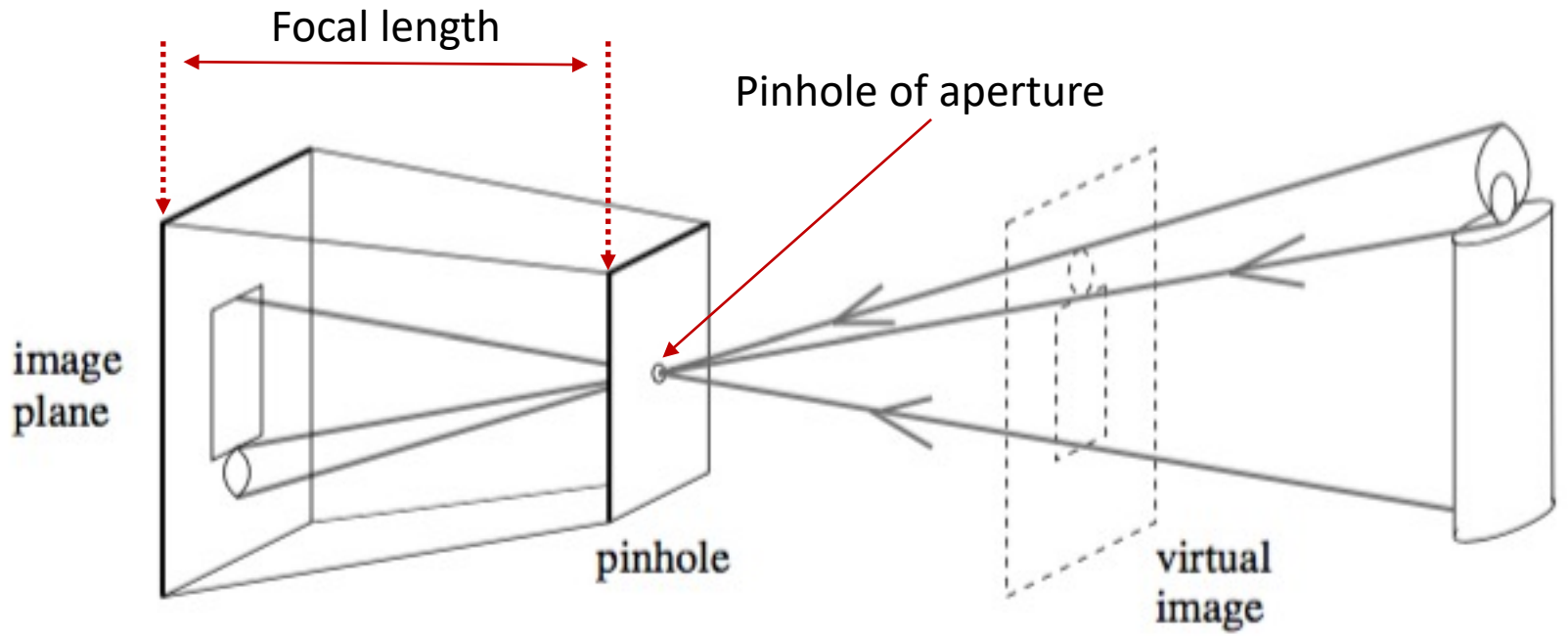
- Pinhole camera: a camera *without a lens* but with a tiny aperture, a *pinhole*

A long history

- Very old idea (several thousands of years BC)
- First clear description from Leonardo Da Vinci (1502)
- Oldest known published drawing of a camera obscura by Gemma Frisius (1544)



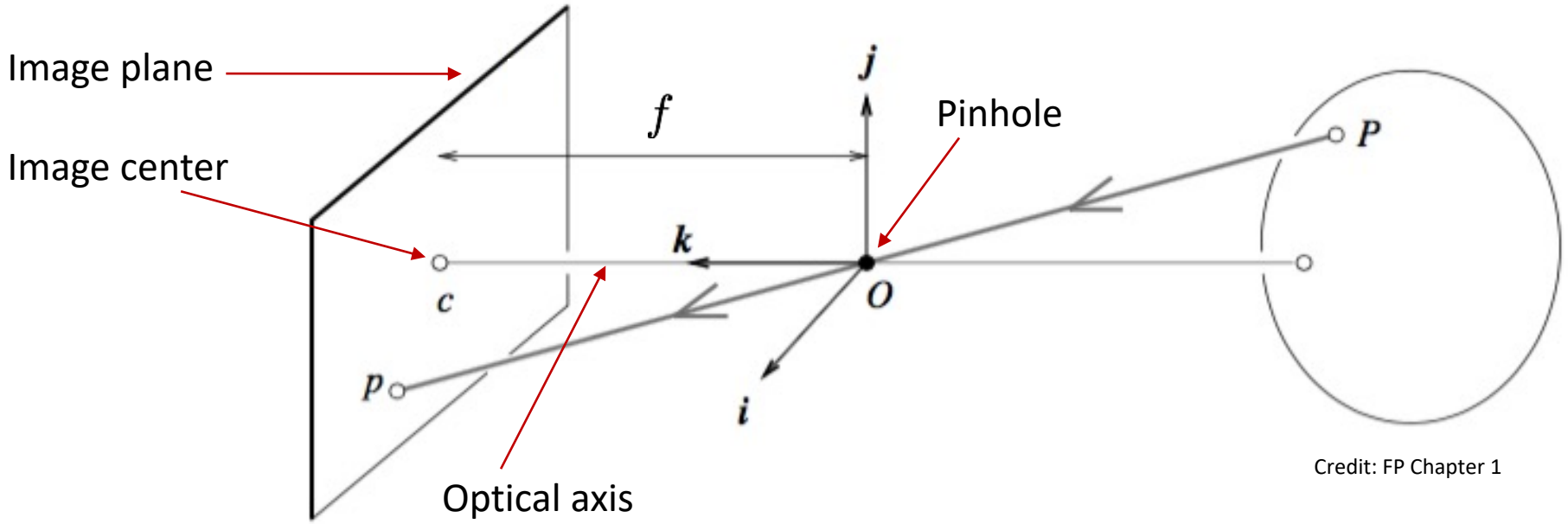
Pinhole camera



Credit: FP Chapter 1

- Perspective projection creates inverted images
- Sometimes it is convenient to consider a *virtual image* associated with a plane lying in front of the pinhole
- Virtual image not inverted but otherwise equivalent to the actual one

Pinhole perspective



Credit: FP Chapter 1

$P = (X, Y, Z)$

Perspective

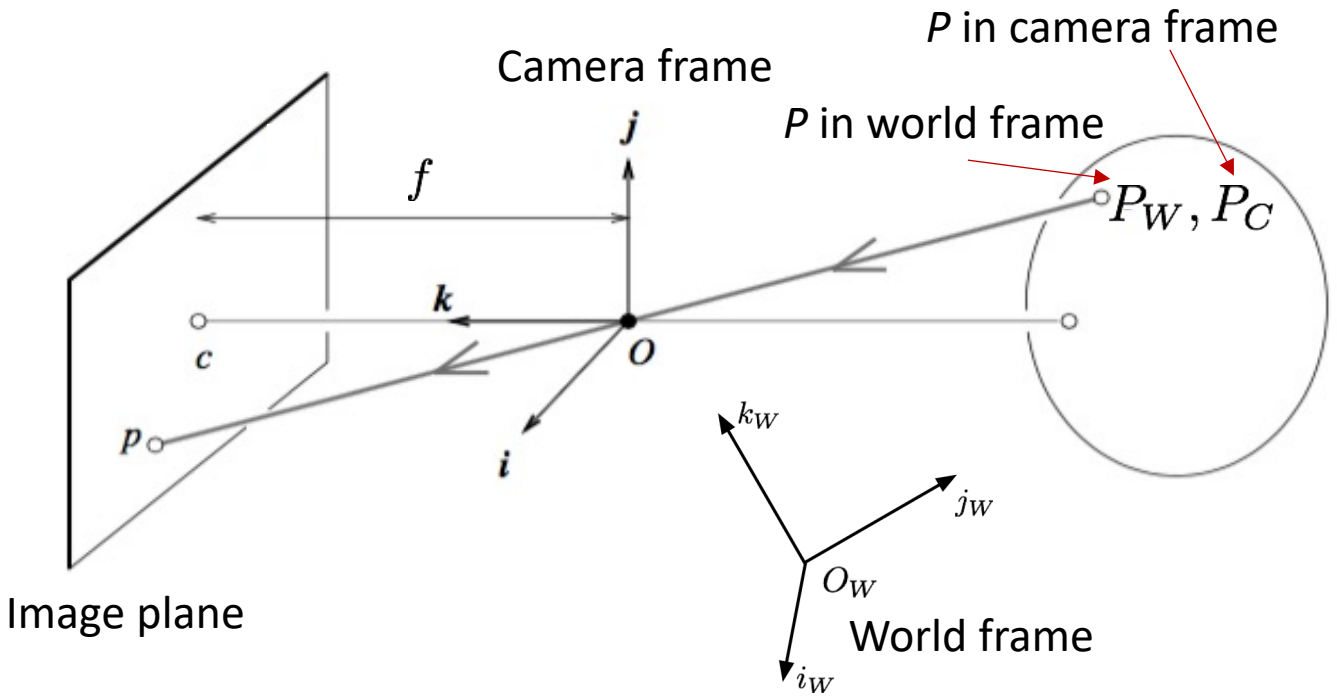
$p = (x, y, z)$

- Since *P*, *O*, and *p* are collinear: $\overline{Op} = \lambda \overline{OP}$ for some $\lambda \in R$
- Also, *z=f*, hence

$$\begin{cases} x = \lambda X \\ y = \lambda Y \\ z = \lambda Z \end{cases} \Leftrightarrow \lambda = \frac{x}{X} = \frac{y}{Y} = \frac{z}{Z} \Rightarrow \begin{cases} x = f\frac{X}{Z} \\ y = f\frac{Y}{Z} \end{cases}$$

Perspective projection

- Goal: find how world points map in the camera image
- Assumption: pinhole camera model (*all results also hold under thin lens model, assuming camera is focused at* ∞)



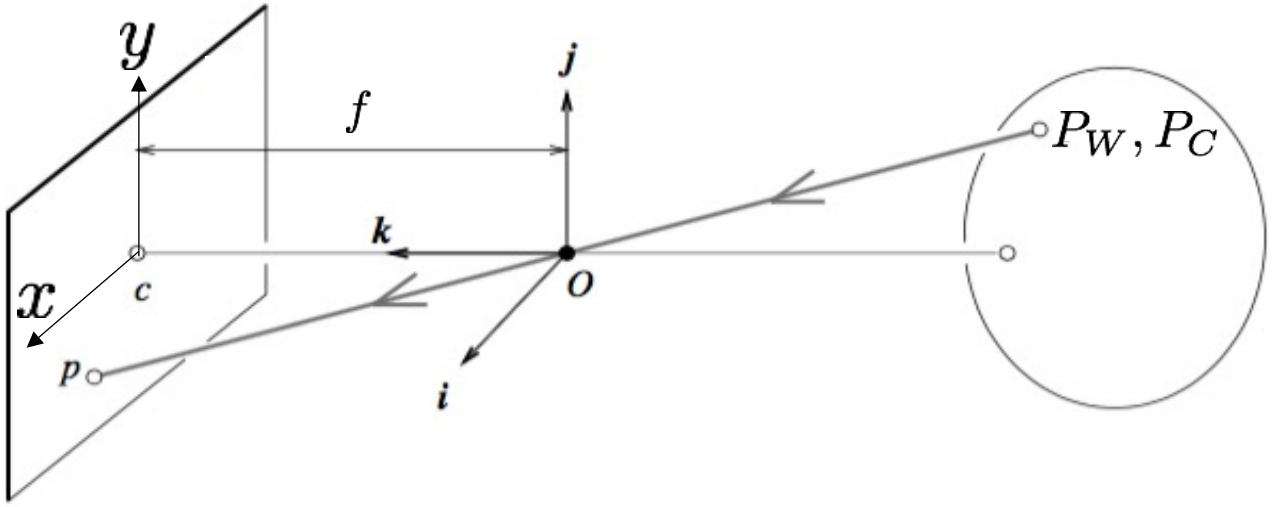
Roadmap:

1. Map P_c into *p* (image plane)
2. Map *p* into (u,v) (pixel coordinates)
3. Transform P_w into P_c

Step 1

- Task: Map $P_c = (X_C, Y_C, Z_C)$ into $p = (x, y)$ (image plane)
- From before

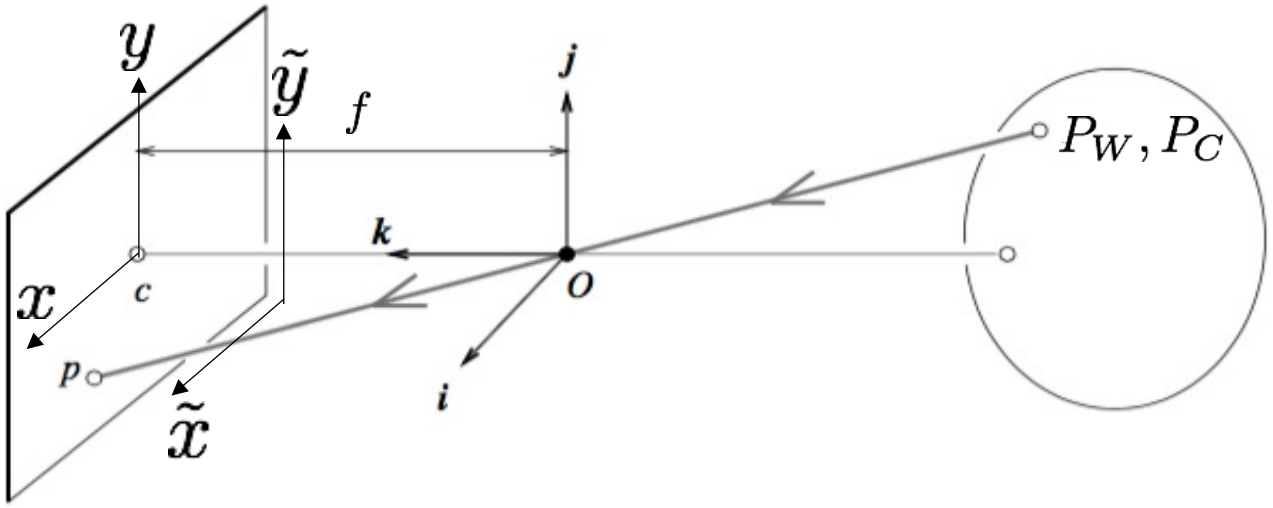
$$\begin{cases} x = f\frac{X_C}{Z_C} \\ y = f\frac{Y_C}{Z_C} \end{cases}$$

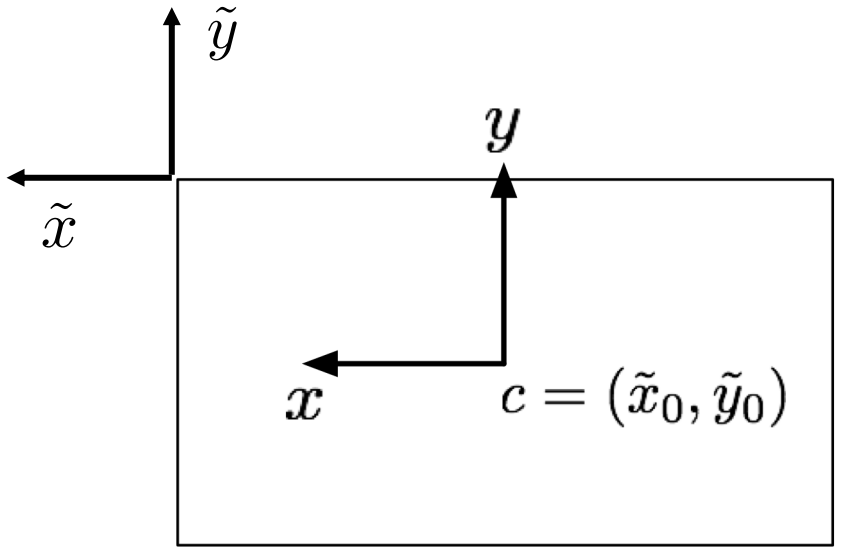


Step 2.a

- Actual origin of the camera coordinate system is usually at a corner (e.g., top left, bottom left)

$$\tilde{x} = f\,\frac{X_C}{Z_C} + \tilde{x}_0, \qquad \tilde{y} = f\,\frac{Y_C}{Z_C} + \tilde{y}_0,$$





Step 2.b

- Task: convert from image coordinates $(\tilde{x}, \tilde{y})$ to pixel coordinates (u, v)
- Let k_x and k_y be the number of pixels per unit distance in image coordinates in the *x* and *y* directions, respectively

$$u = k_x \tilde{x} = \overbrace{k_x f}^{\alpha} \frac{X_C}{Z_C} + \overbrace{k_x \tilde{x}_0}^{u_0}$$

$$v = k_y \tilde{y} = \underbrace{k_y f}_{\beta} \frac{Y_C}{Z_C} + \underbrace{k_y \tilde{y}_0}_{v_0}$$

$$\Rightarrow$$

$$\boxed{\begin{aligned} u &= \alpha \frac{X_C}{Z_C} + u_0 \\ v &= \beta \frac{Y_C}{Z_C} + v_0 \end{aligned}}$$

Nonlinear transformation

Homogeneous coordinates

- Goal: represent the transformation as a linear mapping
- Key idea: introduce homogeneous coordinates

Inhomogenous -> homogeneous

$$\begin{bmatrix} x \\ y \end{bmatrix} \Longrightarrow \lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \qquad \begin{bmatrix} x \\ y \\ z \end{bmatrix} \Longrightarrow \lambda \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Homogenous -> inhomogeneous

$$\begin{bmatrix} x\lambda \\ y\lambda \\ \lambda \end{bmatrix} \Longrightarrow \begin{bmatrix} x \\ y \end{bmatrix} \qquad \begin{bmatrix} x\lambda \\ y\lambda \\ z\lambda \\ \lambda \end{bmatrix} \Longrightarrow \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

Perspective projection in homogeneous coordinates

- Projection can be equivalently written in homogeneous coordinates

$$\begin{bmatrix} \alpha & 0 & u_0 & 0 \\ 0 & \beta & v_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{pmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{pmatrix} = \begin{pmatrix} \alpha X_c + u_0 Z_c \\ \beta Y_c + v_0 Z_c \\ Z_c \end{pmatrix}$$

K

Camera matrix/
Matrix of intrinsic parameters

P_c in homogeneous coordinates

Homogeneous pixel coordinates

- In homogeneous coordinates, the mapping is linear:

$$p^h = [K \quad 0_{3\times 1}] P_C^h$$

Point *p* in homogeneous pixel coordinates

Point P_c in homogeneous camera coordinates

Skewness

- In some (rare) cases

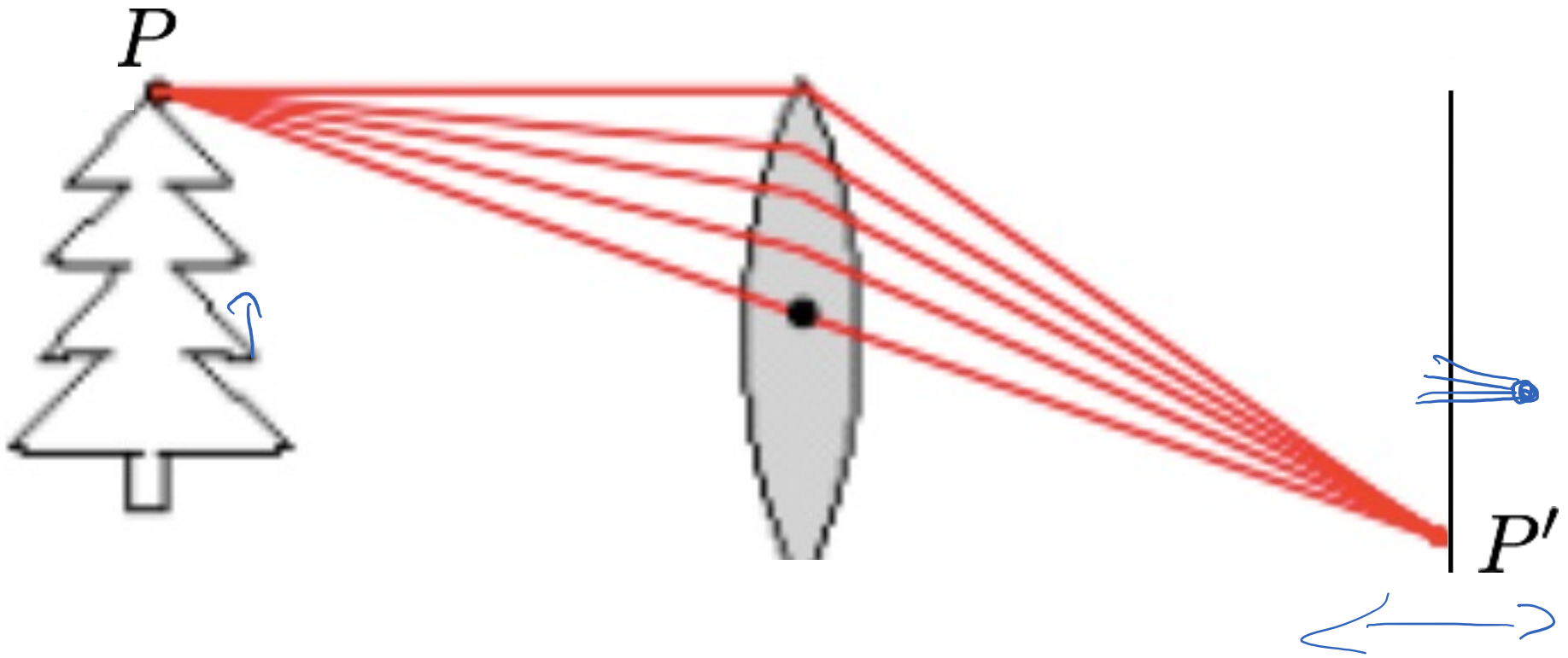
Skew parameter

$$K = \begin{bmatrix} \alpha & \gamma & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

- When is $\gamma \neq 0$?
  - x- and y-axis of the camera are not perpendicular (unlikely)
  - For example, as a result of taking an image of an image
- Five parameters in total!

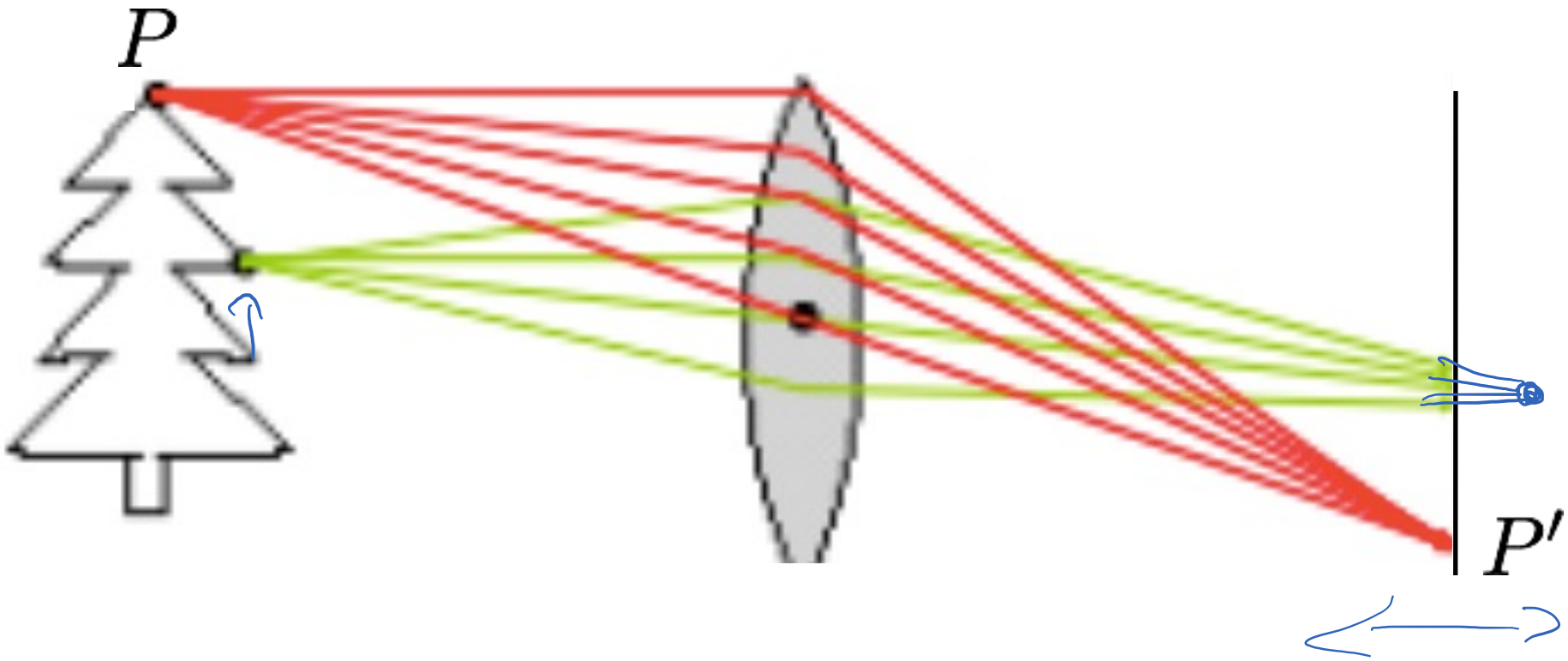
Lenses

- Lens: an optical element that focuses light by means of refraction



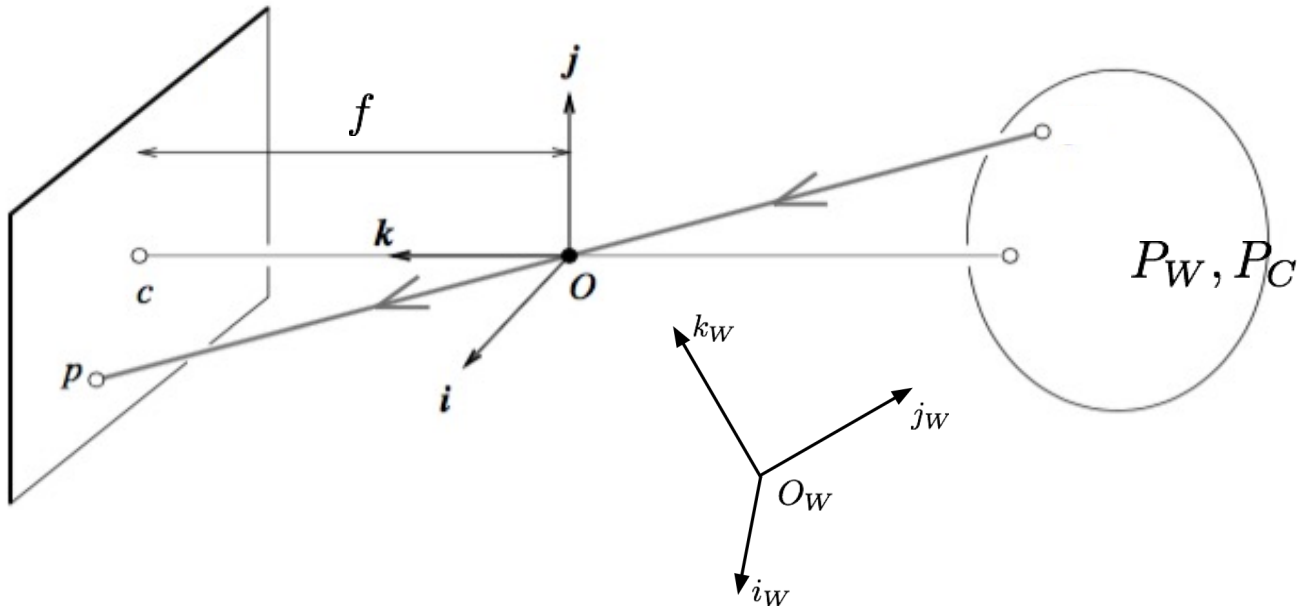
Lenses

- Lens: an optical element that focuses light by means of refraction

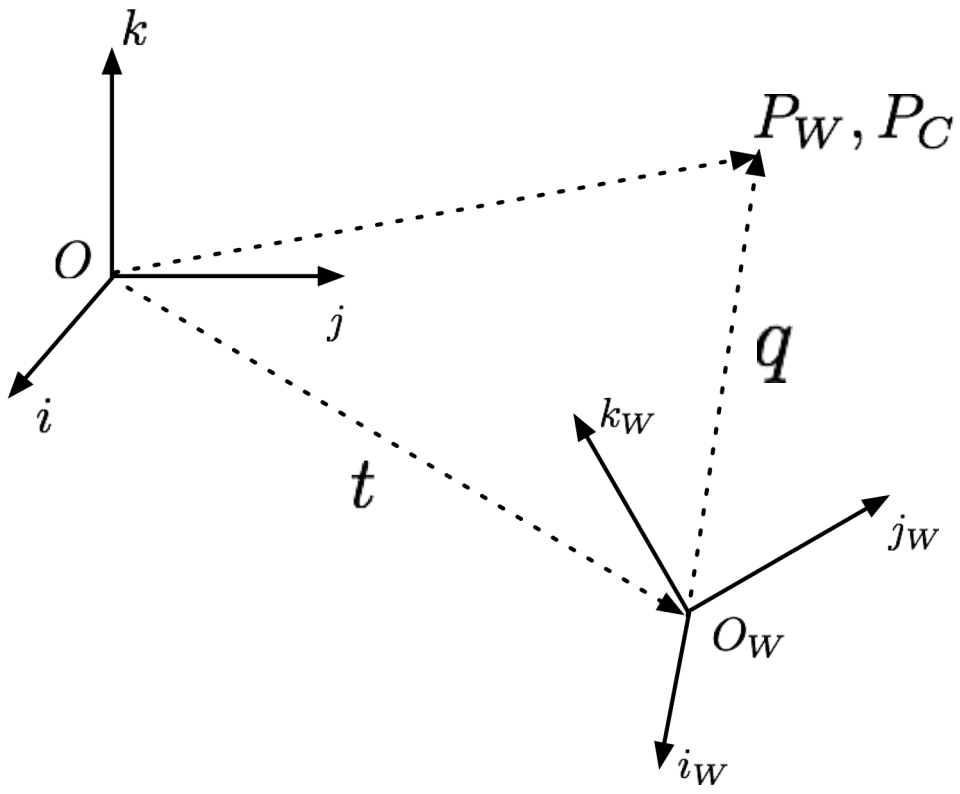


Step 3

- Last step is to include in our mapping an additional transformation to account for the difference between the world frame and the 3D camera reference frame



Rigid body transformations



$$P_C = t + q$$

$$q = R\,P_W$$

where (*R, t)* is the rigid body transform from world to camera frame

$$R = \begin{bmatrix} i_W \cdot i & j_W \cdot i & k_W \cdot i \\ i_W \cdot j & j_W \cdot j & k_W \cdot j \\ i_W \cdot k & j_W \cdot k & k_W \cdot k \end{bmatrix}$$

$$\Rightarrow P_C = t + R\,P_W$$

Rigid transformations in homogeneous coordinates

$$\begin{pmatrix} P_C \\ 1 \end{pmatrix} = \begin{bmatrix} R & t \\ 0_{1\times 3} & 1 \end{bmatrix} \begin{pmatrix} P_W \\ 1 \end{pmatrix}$$

Point P_c in homogeneous coordinates

Point P_w in homogeneous coordinates

Perspective projection equation

- Collecting all results

$$p^h = [K \quad 0_{3\times1}]P_C^h = K[I_{3\times3} \quad 0_{3\times1}] \begin{bmatrix} R & t \\ 0_{1\times3} & 1 \end{bmatrix} P_W^h$$

- Hence

Projection matrix *M*

$$p^h = K[R \quad t]P_W^h$$

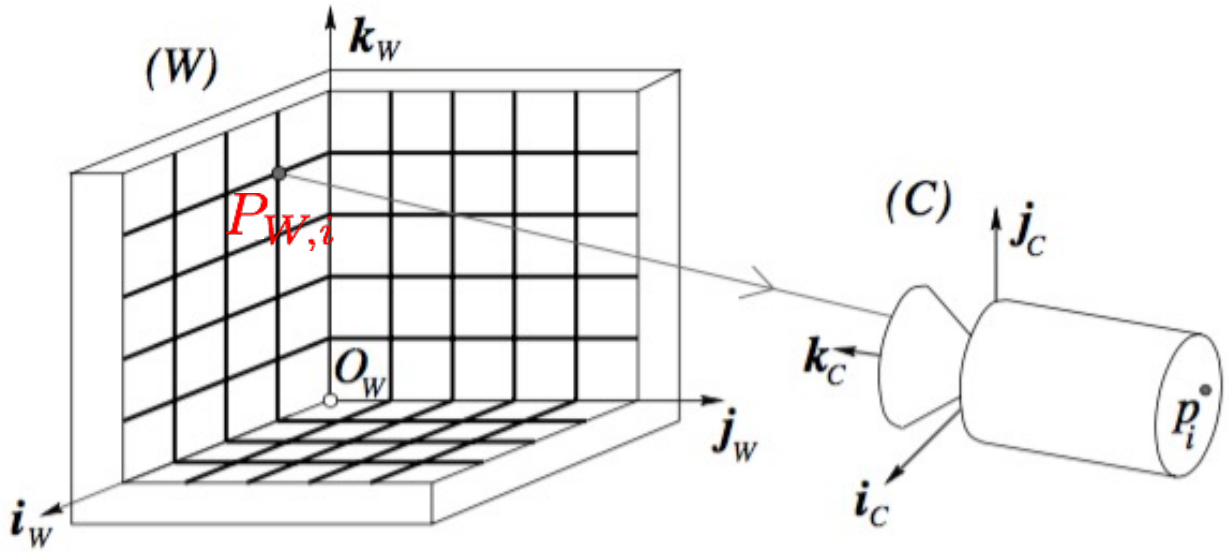
Intrinsic parameters

Extrinsic parameters (transform from world to camera, inverse of camera pose)

- Degrees of freedom: 4 for *K* (or 5 if we also include skewness), 3 for *R*, and 3 for *t*. Total is 10 (or 11 if we include skewness)

Camera calibration: direct linear transformation method

- Goal: find the intrinsic and extrinsic parameters of the camera

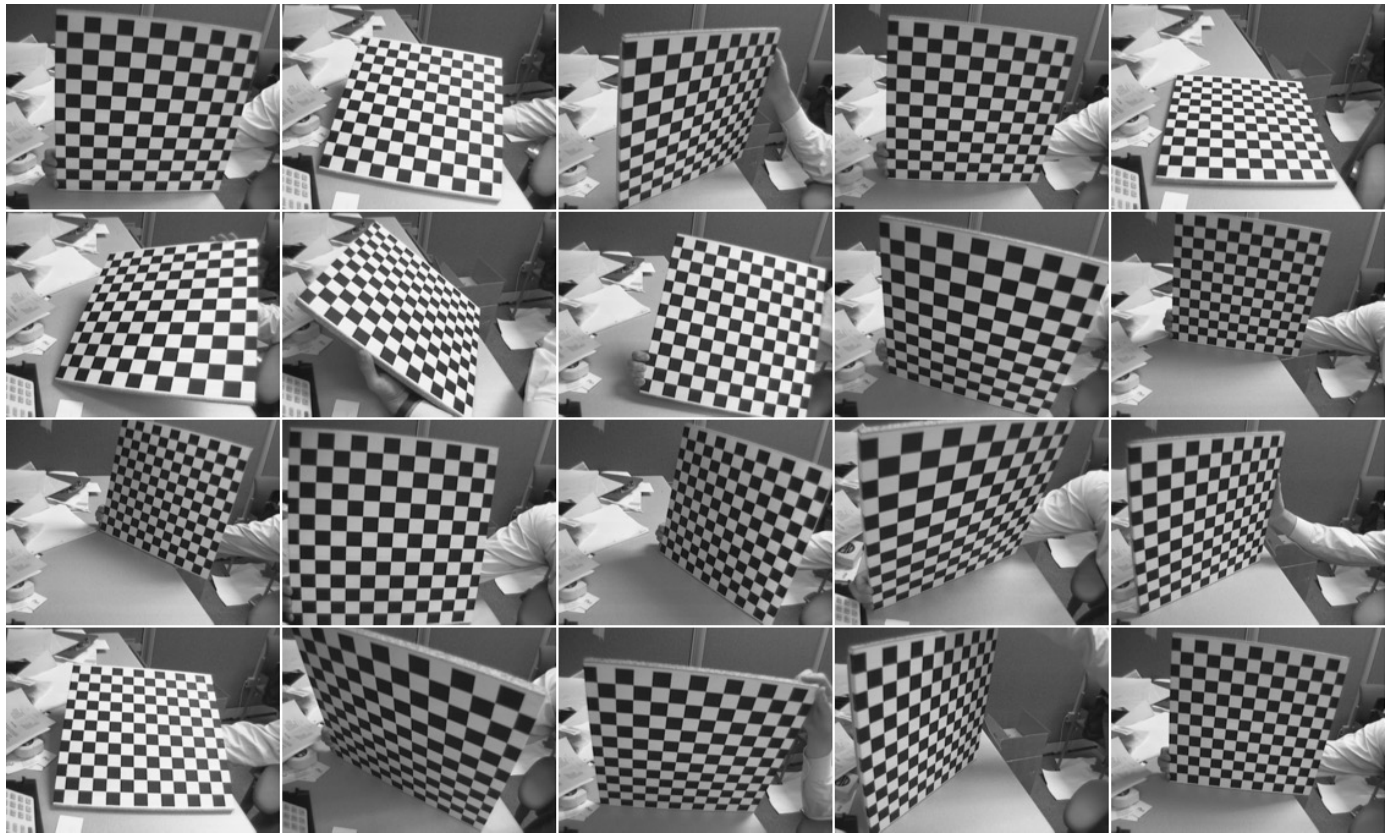


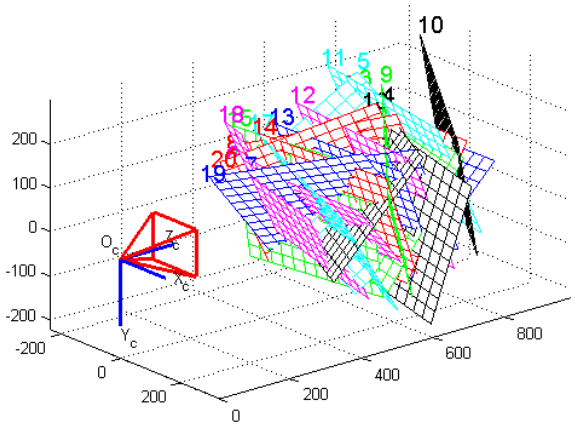
Strategy: given known correspondences $p_i \leftrightarrow P_{W,i}$, compute unknown parameters *K*, *R*, *t* by applying perspective projection

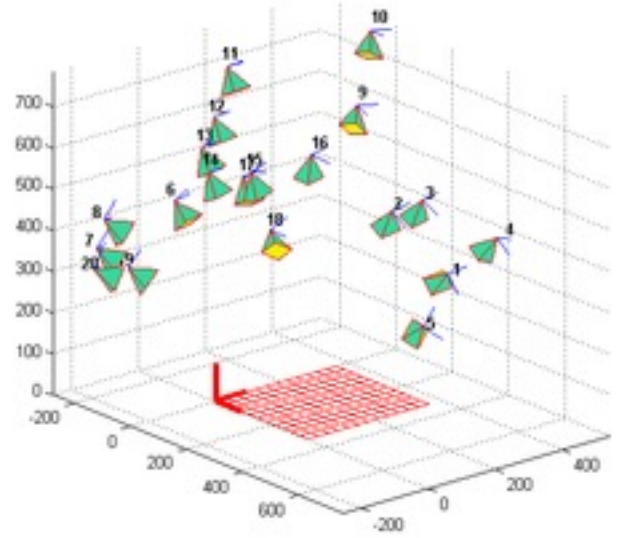
$P_{W,1}, P_{W,2}, \ldots, P_{W,n}$ with known positions in world frame

$p_1, p_2, \ldots, p_n$ with known positions in image frame

Examples for Calibration Images







- Due to co-planarity of cal board, need multiple images from different poses
- Origin of world frame defined by board lower right corner

Source: Wikipedia

Step 1

- First consider combined parameters

1×4 vector

$$p_i^h = M\, P_{W,i}^h,\ \ i = 1, \ldots, n, \qquad \text{where} \quad M = K[R \quad t] = \begin{bmatrix} m_1 \\ m_2 \\ m_3 \end{bmatrix}$$

- This gives rise to $2n$ component-wise equations, for $i = 1, \ldots, n$

$$u_i = \frac{m_1 \cdot P_{W,i}^h}{m_3 \cdot P_{W,i}^h} \qquad \text{or} \qquad u_i \left(m_3 \cdot P_{W,i}^h\right) - m_1 \cdot P_{W,i}^h = 0$$

$$v_i = \frac{m_2 \cdot P_{W,i}^h}{m_3 \cdot P_{W,i}^h} \qquad\qquad v_i \left(m_3 \cdot P_{W,i}^h\right) - m_2 \cdot P_{W,i}^h = 0$$

Calibration problem

- Stacking all equations together

$$\tilde{P}m = 0, \qquad \text{where } m = \begin{bmatrix} m_1^T \\ m_2^T \\ m_3^T \end{bmatrix}$$

$2n$ x 12 matrix of known coefficients

12 x 1 vector of unknown coefficients

12 x 1

- $\tilde{P}$ contains in block form the known coefficients stemming from the given correspondences
- To estimate 11 coefficients, we need at least 6 correspondences
- Due to co-planarity of cal board, need multiple images from different poses

Solution

- To find non-zero solution

$$\min_{m \in R^{12}} \quad \|\tilde{P}m\|^2$$
$$\text{subject to} \quad \|m\|^2 = 1$$

- Solution: select eigenvector of $\tilde{P}^T \tilde{P}$ with the smallest eigenvalue
- Readily computed via SVD (singular value decomposition)

Step 2

- Next, we need to extract the camera parameters, i.e., we want to factorize *M* as

$$M = [KR \quad Kt]$$

- Use "RQ" factorization (a variant of QR factorization): the submatrix $M_{1:3,1:3}$ is decomposed into the product of an upper triangular matrix *K* and a rotation matrix *R*.

- We compute t from $t = K^{-1}(Kt)$ where (Kt) is the last column of M.

- Calibration will be investigated in Problem 1 in HW3

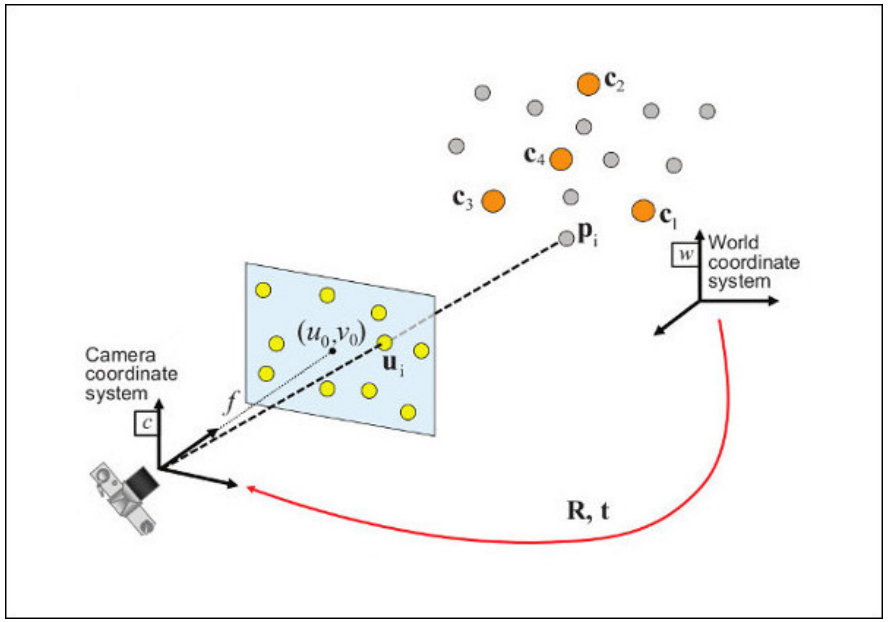
Perspective-n-Point (PnP) Problem

- Suppose we know camera calibration matrix K, and we have a known 3D model of an object in the scene (e.g., a CAD model)
- Suppose we can find point-wise correspondences between image pixels and 3D points on the CAD model
- Find the pose of the object (R, t) with respect to the camera
- Camera calibration already gives a solution!
- More efficient algorithms exist using knowledge of K and structure of R

Projection matrix *M*

$$p^h = K[R \quad t]P_W^h$$

Find R and t



Radial distortion

- So far, we have assumed that a linear model is an accurate model of the imaging process
- For real (non-pinhole) lenses this assumption will not hold



No distortion



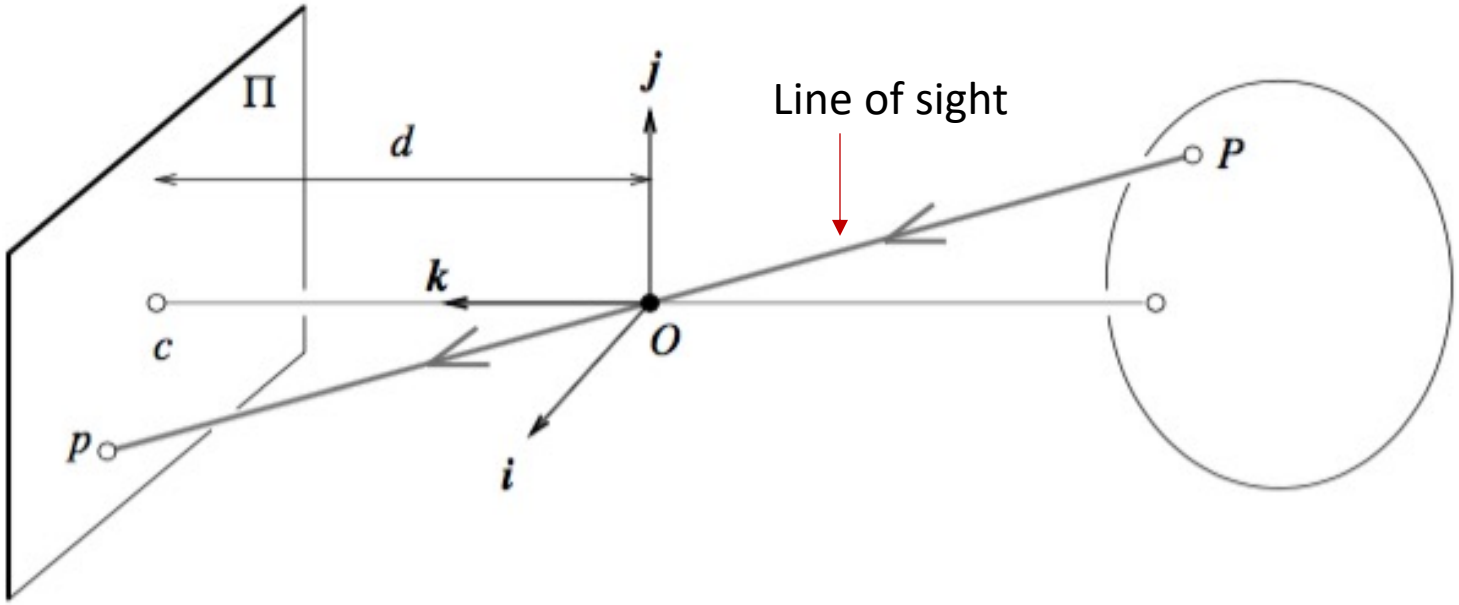
Barrel distortion



Pincushion distortion

Credit: SNS

Measuring depth



$$p^h = K[R \quad t]P_W^h$$

Homogeneous coordinates

Once the camera is calibrated, can we measure the location of a point *P* in 3D given its known observation *p*?

- No: one can only say that *P* is located *somewhere* along the line joining *p* and *O*!

Issues with recovering structure (3D point locations in world frame)

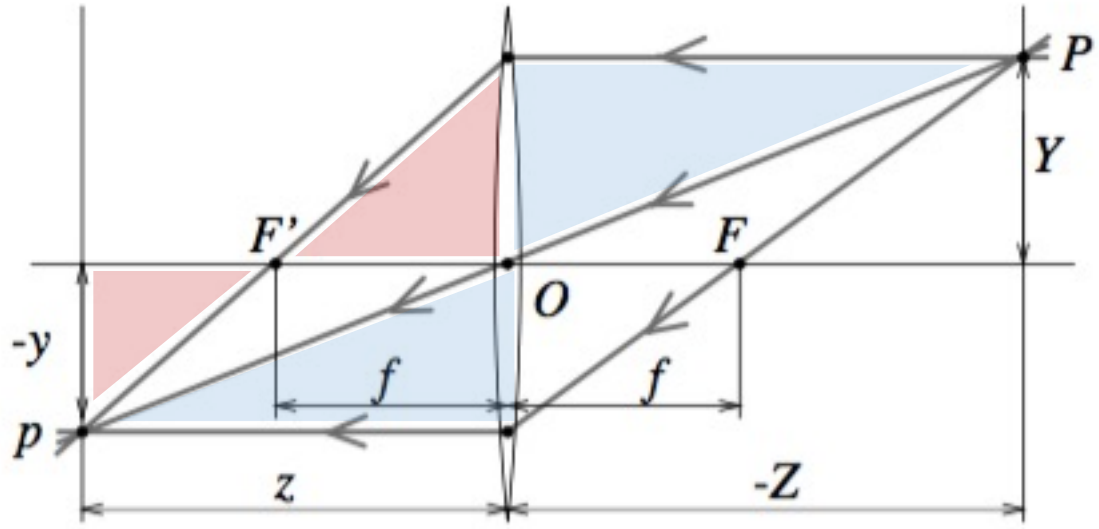


Recovering structure

- Structure: 3D scene to be reconstructed by having access to 2D images

- Common methods
  1. Through recognition of landmarks (e.g., orthogonal walls)
  2. Depth from focus: determines distance to one point by taking multiple images with better and better focus
  3. Stereo vision: processes two distinct images taken at the *same time* and assumes that the relative pose between the two cameras is *known*
  4. Structure from motion: processes two images taken with the same or different cameras at *different times* and from different *unknown* positions

Depth from focus



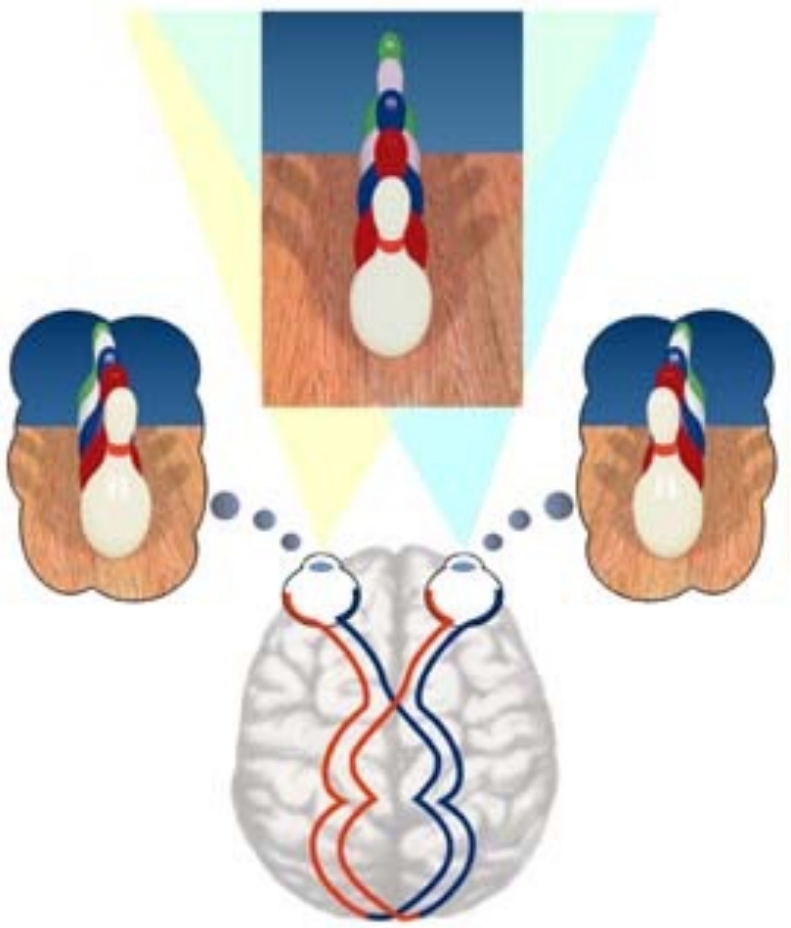
Credit: FP Chapter 1

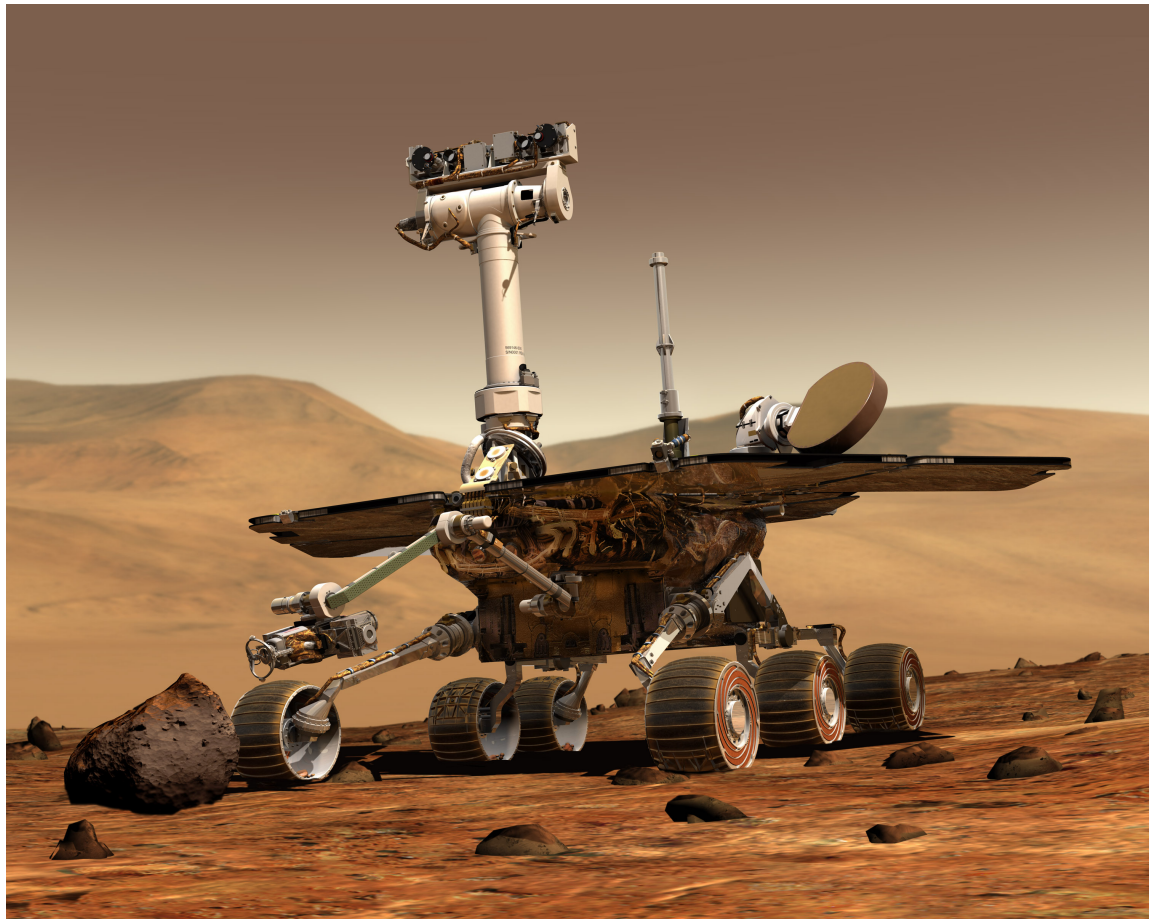
$$\Rightarrow \frac{1}{z} + \frac{1}{Z} = \frac{1}{f}$$

Thin lens equation

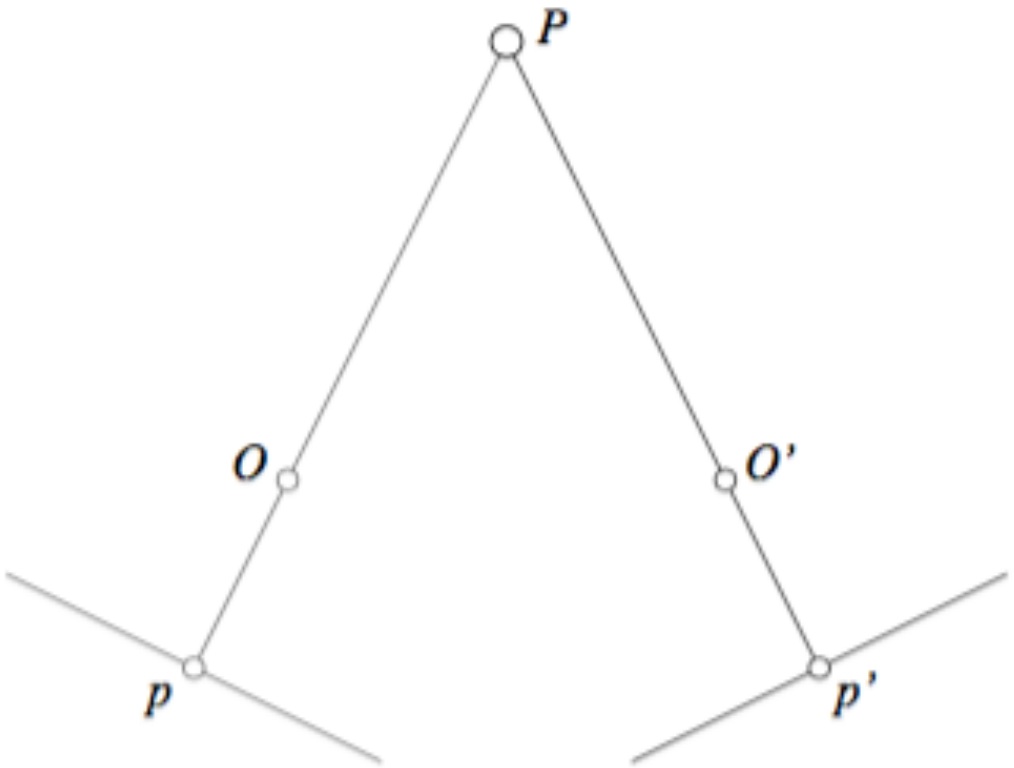
- Take several images until the projection of point *P* is in focus; let *z* denote the distance at which the image is in focus
- Since we know *z* and *f,* through the thin lens equation we obtain *Z*

Stereopsis, or why we have two eyes



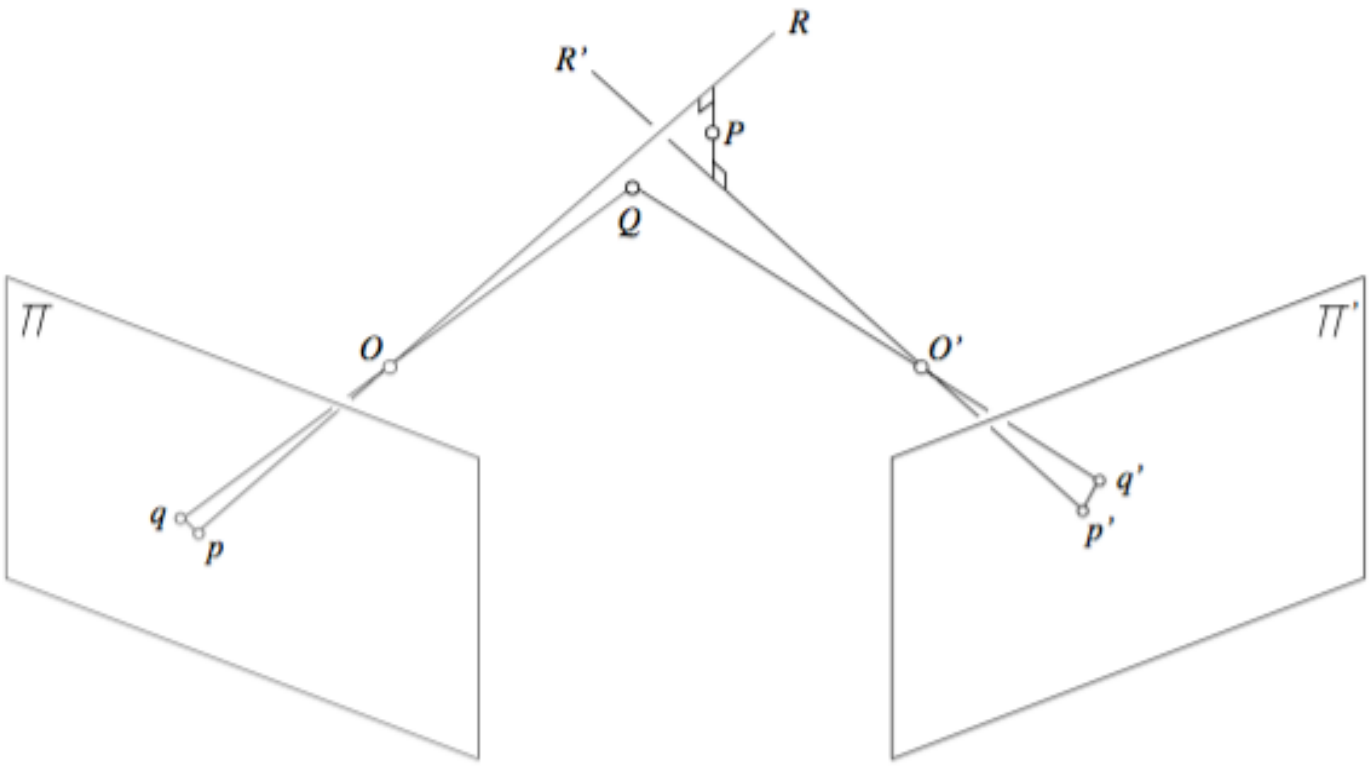


Binocular reconstruction



- Given: *calibrated* stereo rig and two image matching points p and p'
- Find corresponding scene point by intersecting the two rays $\overline{Op}$ and $\overline{O'p'}$ (process known as triangulation)

Approximate triangulation



- Due to noise, triangulation problem is often solved as finding the point Q with images q and q' that minimizes

$$\underbrace{d^2(p,q) + d^2(p',q')}_{\text{Re-projection error}}$$

Next time: Structure from Motion (SfM)

